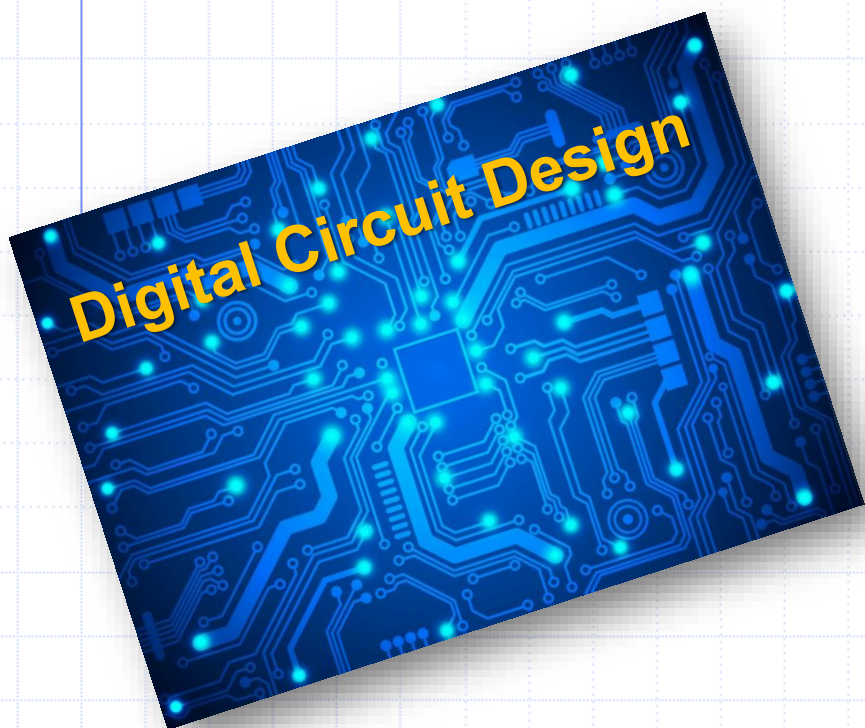


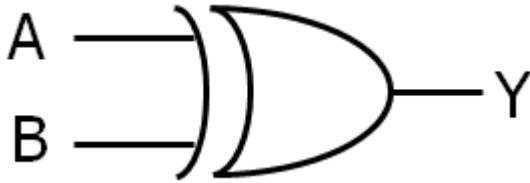


طراحی مدارهای منطقی

فصل چهارم:
مدارهای ترکیبی

University of Tabriz
Department of Computer Science
Jafar Razmara

گیت Xor



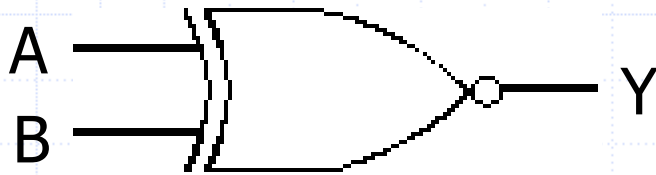
$$Y = AB' + A'B$$

$$1 \oplus 1 = 0$$

$$1 \oplus 0 = 1$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

گیت Xnor



$$Y = (AB' + A'B)'$$
$$= AB + A'B'$$

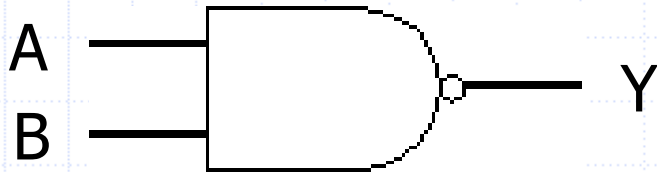
$$1 \odot 1 = 1$$

$$1 \odot 0 = 0$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

گیت Nand

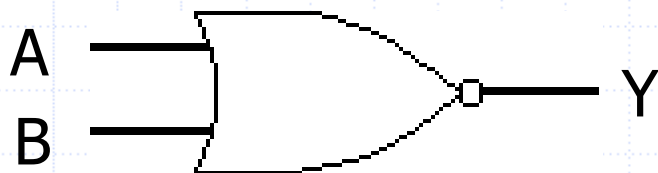
- گیت Nand ترکیبی از گیت and و گیت not می باشد.



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

گیت Nor

- گیت Nor ترکیبی از گیت or و گیت not می باشد.



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

پیاده‌سازی توابع با استفاده گیت‌های Nor و Nand

- هر تابع جبری با هر شکل و اندازه‌ای با استفاده از یک جدول درستی قابل نمایش است؛ و به فرم ۲ طبقه ای از گیت‌های and-or و یا or-and می‌توان پیاده‌سازی کرد.
- با توجه به اینکه گیت‌های Nor و Nand قابلیت پیاده‌سازی سایر گیت‌ها را دارند؛ بنابراین می‌توان هر مداری را با استفاده از این گیت‌ها پیاده‌سازی کرد.

پیاده‌سازی توابع با استفاده از گیت Nand

- با استفاده از گیت Nand می‌توان عملکرد سه گیت پایه and، or و not را پیاده‌سازی کرد:

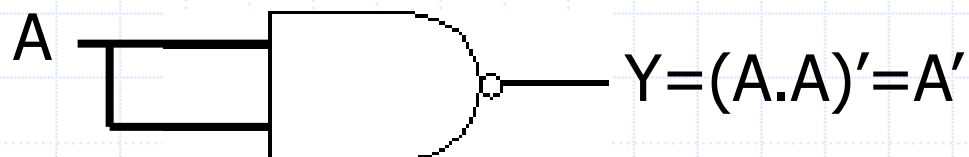
$$\text{NAND}(a,a) = (aa)' = a' = \text{NOT}(a)$$

$$\text{NAND}'(a,b) = (ab)'' = ab = \text{AND}(a,b)$$

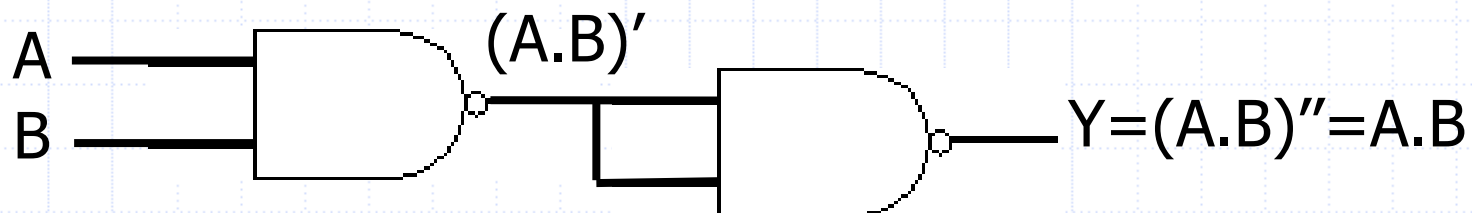
$$\text{NAND}(a',b') = (a'b')' = a+b = \text{OR}(a,b)$$

پیاده‌سازی توابع با استفاده از گیت Nand

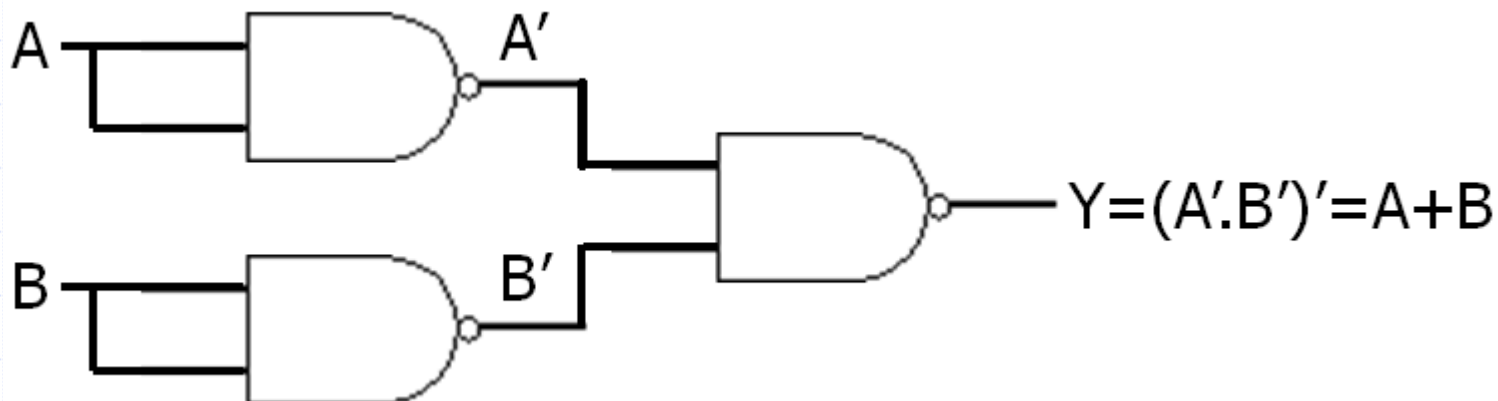
**NOT
Gate**



**AND
Gate**



**OR
Gate**



پیاده‌سازی توابع با استفاده از گیت Nor

- با استفاده از گیت Nor می‌توان عملکرد سه گیت پایه and، or و not را پیاده‌سازی کرد:

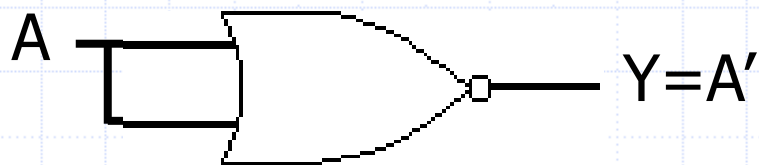
$$\text{NOR}(a,a) = (aa)' = a' = \text{NOT}(a)$$

$$\text{NOR}'(a,b) = (a+b)'' = a+b = \text{OR}(a,b)$$

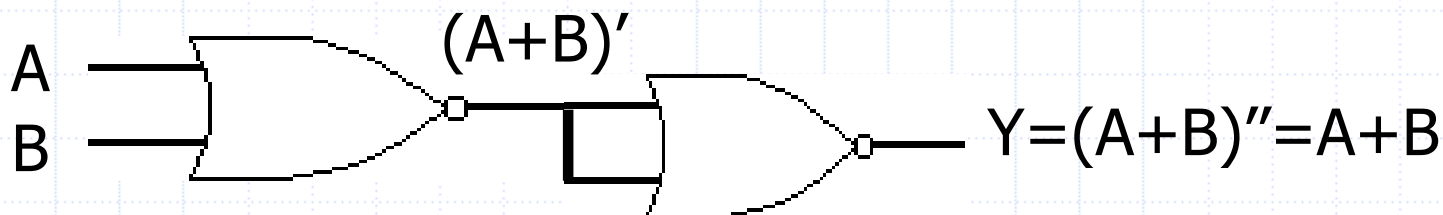
$$\text{NOR}(a',b') = (a'+b')' = ab = \text{AND}(a,b)$$

پیاده‌سازی توابع با استفاده از گیت Nor

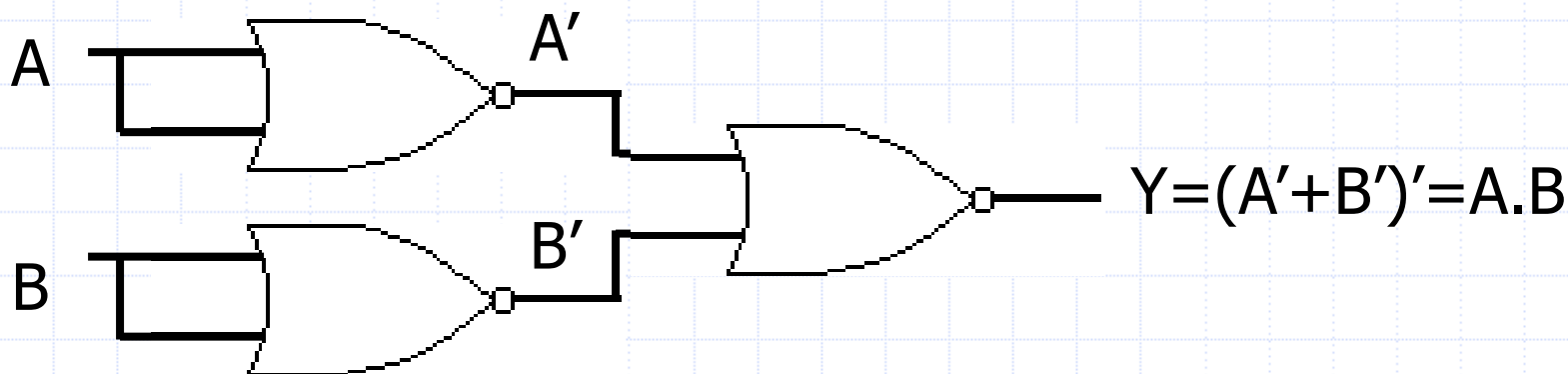
**NOT
Gate**



**OR
Gate**



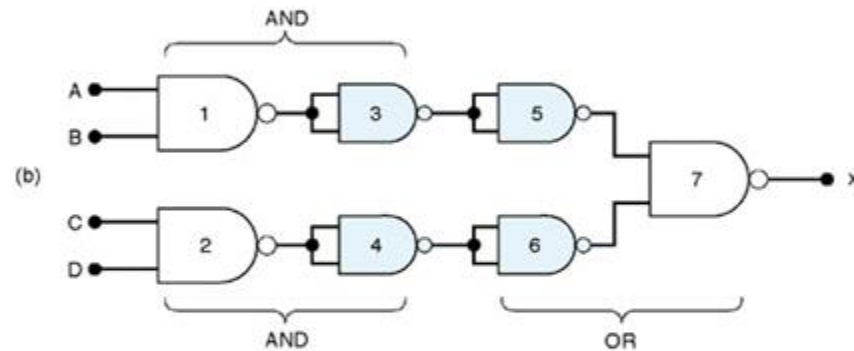
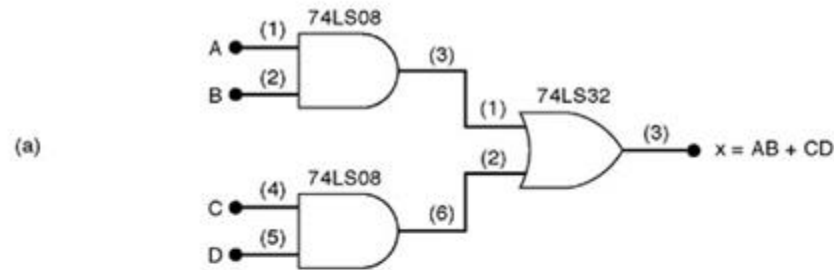
**AND
Gate**



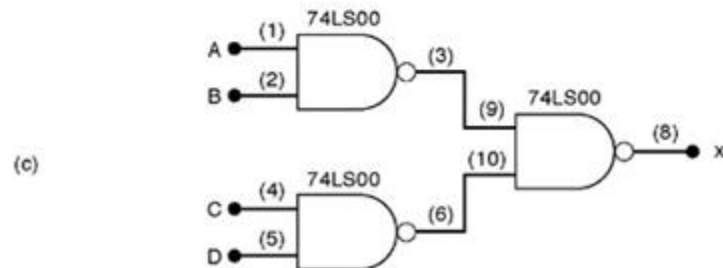
پیاده‌سازی توابع با استفاده از گیت‌های Nand و Nor

- گیت‌های Nand و Nor از لحاظ تابعی کامل هستند یعنی هر تابعی شامل ترکیبی از گیت‌های مختلف را می‌توان با استفاده از یکی از این گیت‌ها پیاده‌سازی کرد.
- ساخت یک IC شامل یکی از گیت‌های Nand و یا Nor ساده‌تر از ساخت یک IC شامل ترکیبی از گیت‌های and، or و Not می‌باشد.
- همچنین، گیت‌های Nand و Nor سریع‌تر عمل کرده و ساخت آنها کم هزینه‌تر می‌باشد.

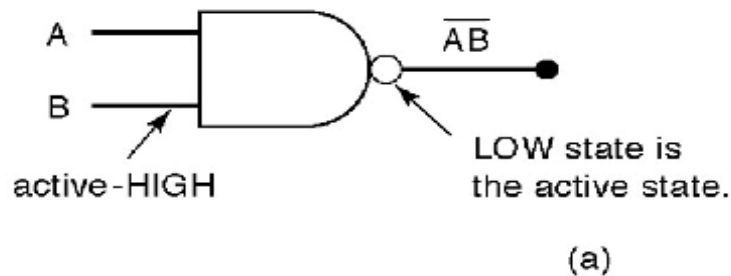
پیاده‌سازی توابع با استفاده از گیت Nand



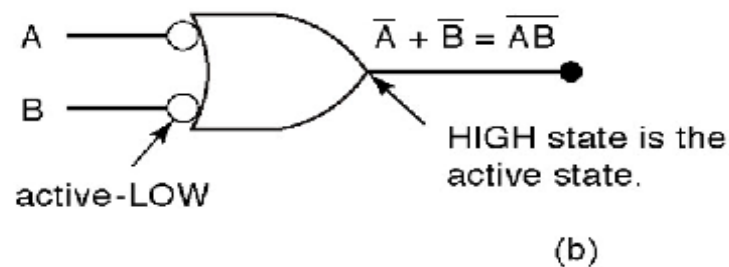
After eliminating double inversions



پیاده‌سازی توابع با استفاده از گیت Nand

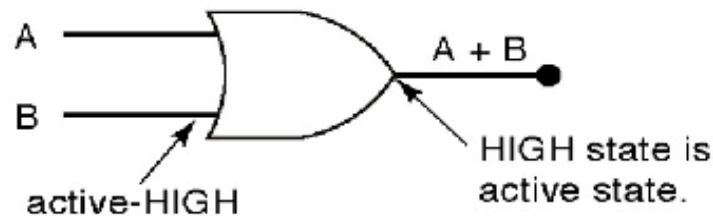


Output goes LOW only when all inputs are HIGH.



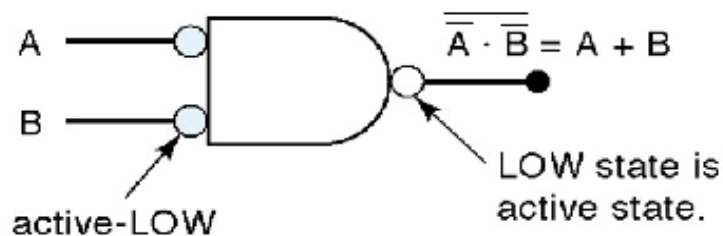
Output is HIGH when any input is LOW.

پیاده‌سازی توابع با استفاده از گیت Nand



(a)

Output goes HIGH when any input is HIGH.



(b)

Output goes LOW only when all inputs are LOW.

پیاده‌سازی توابع با استفاده از گیت Nand

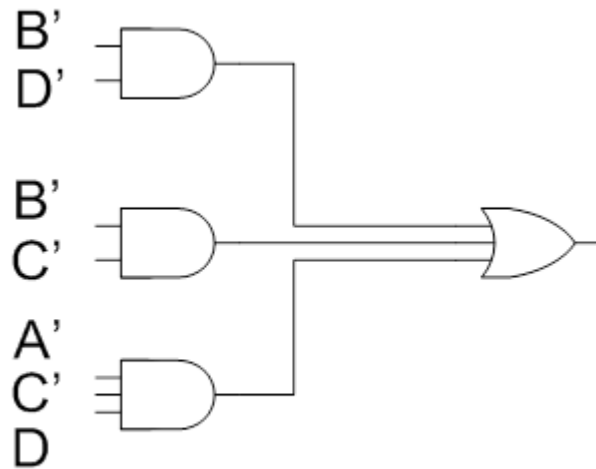
پیاده‌سازی بوسیله گیت‌های NAND:

- ۱- تابع را ساده کرده و آنرا به صورت جمع حاصلضرب ها می‌نویسیم.
- ۲- برای هر جمله حاصلضرب موجود در تابع که حداقل دارای ۲ متغیر می‌باشد یک گیت NAND قرار داده و متغیرهای هر جمله را بصورت ورودی گیت مربوطه در نظر می‌گیریم. این مجموعه، طبقه اول را تشکیل می‌دهد.
- ۳- در طبقه دوم، یک گیت NAND قرار می‌دهیم و خروجی‌های طبقه اول را به ورودی آن وصل می‌کنیم.
- ۴- اگر تابع دارای جمله تک متغیره باشد برای آن در طبقه اول یک گیت Not (در واقع با NAND ی که دو ورودی اش به هم متصل است) قرار می‌دهیم.

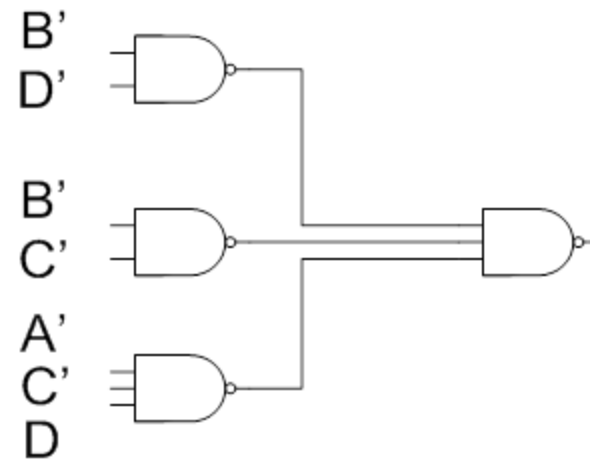
پیاده‌سازی توابع با استفاده از گیت Nand

مثال:

$$F = B'C' + B'D' + C'DA'$$



AND-OR



NAND-NAND

پیاده‌سازی توابع با استفاده از گیت Nor

پیاده‌سازی با استفاده از گیت‌های NOR:

گیت NOR دوگان تابع NAND است.

۱- تابع را ساده کرده و آنرا بصورت ضرب حاصلجمع‌ها می‌نویسیم.

۲- برای هر جمله حاصلجمع موجود در تابع که حداقل ۲ متغیر دارد یک گیت NOR قرار

می‌دهیم. ورودی NOR، متغیرهای حاصلجمع می‌باشد. این گیت‌ها طبقه اول را تشکیل می‌دهند.

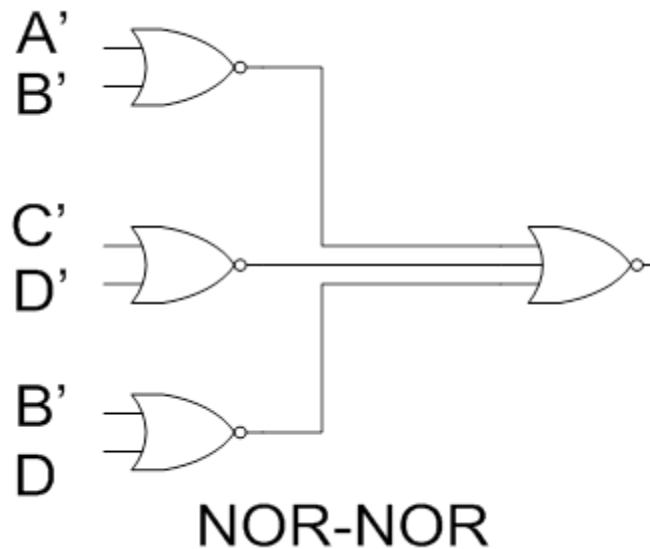
۳- در طبقه دوم، یک گیت NOR قرار می‌دهیم که ورودی آن از خروجی گیت‌های طبقه اول می‌آید.

۴- جمله تک متغیره در طبقه اول نیازمند یک گیت NOT (در واقع NOR ی که دو ورودی اش به هم متصل است) است.

پیاده‌سازی توابع با استفاده از گیت Nor

مثال :

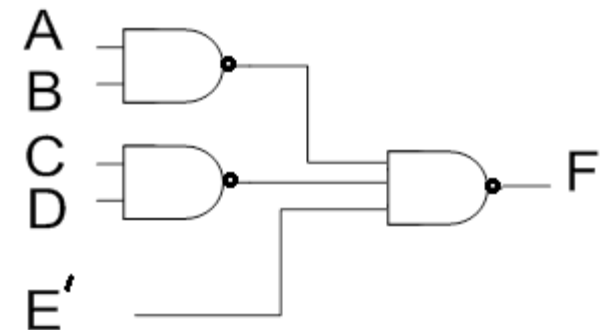
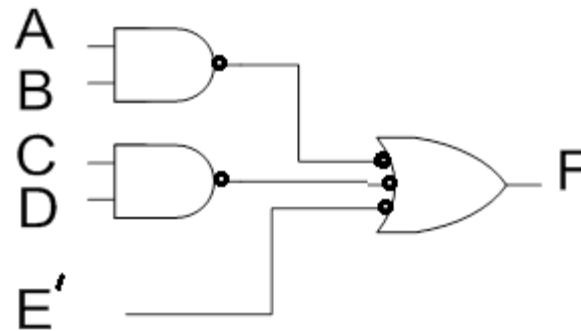
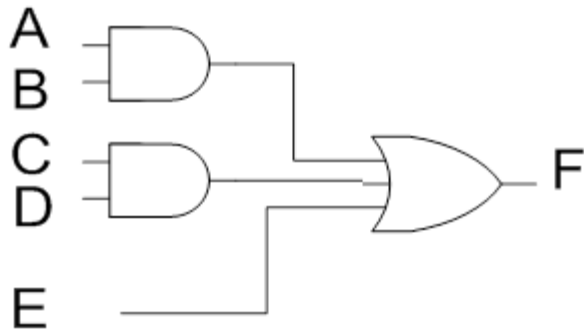
$$F = (A' + B')(C' + D')(B' + D)$$



پیاده‌سازی توابع با استفاده از گیت Nand

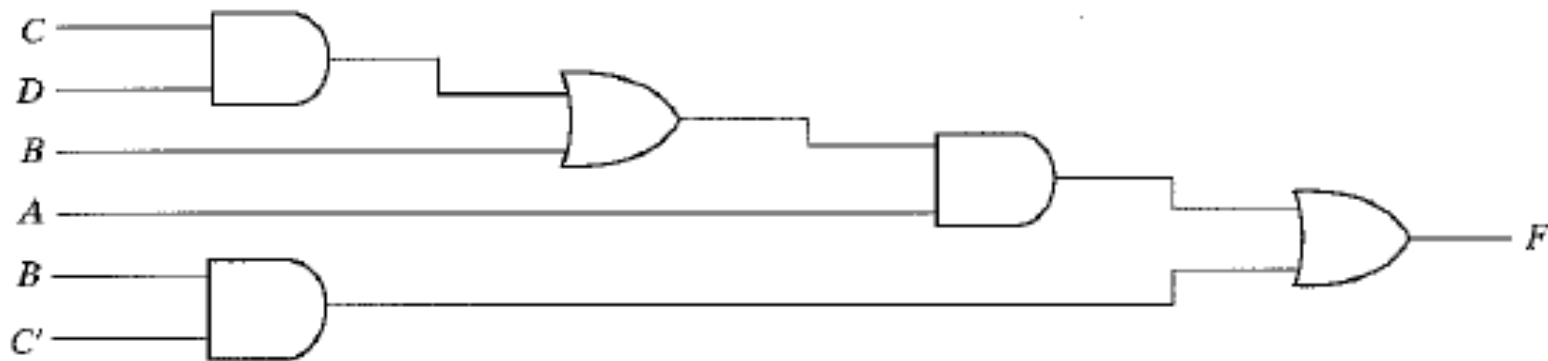
مثال:

$$F = AB + CD + E$$

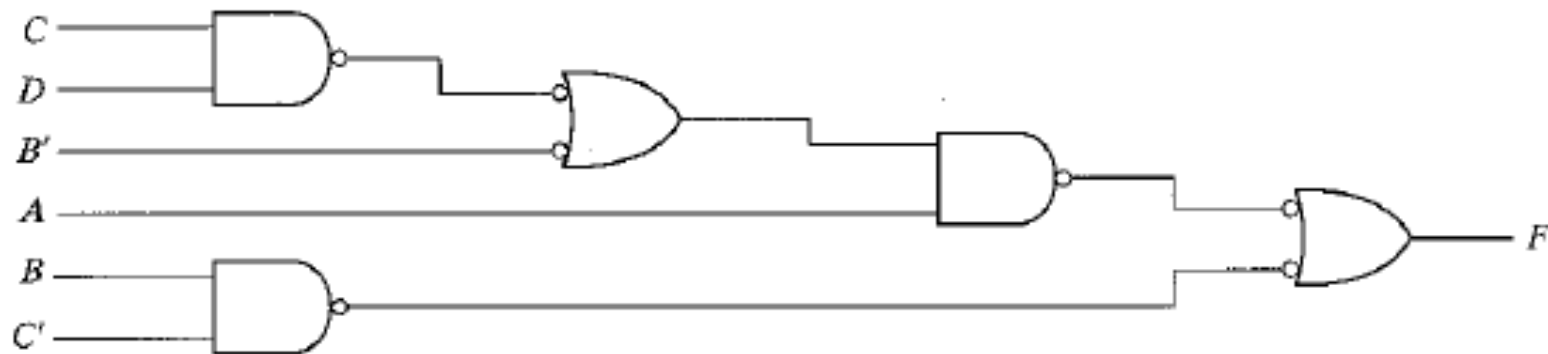


پیاده‌سازی توابع با استفاده از گیت Nand

مثال:



(a) AND-OR gates



(b) NAND gates

FIGURE 3-22

Implementing $F = A(CD + B) + BC'$

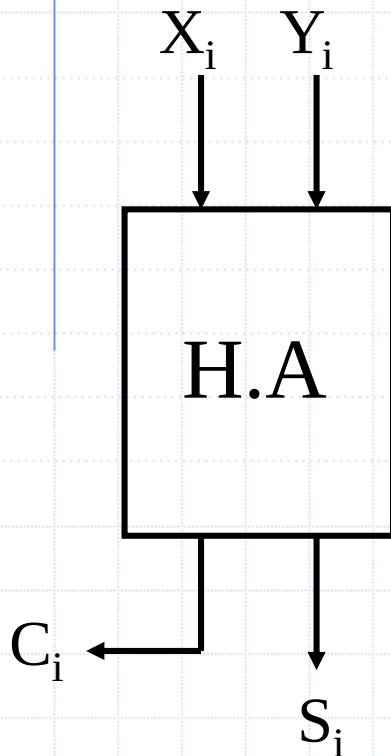
طراحی مدار

مراحل مورد نیاز برای طراحی یک مدار:

- (1) تعیین تعداد بیت های ورودی و خروجی مدار (Interface)
- (2) رسم جدول درستی (Truth Table)
- (3) بدست آوردن یک تابع برای خروجی
- (4) ساده سازی توابع بدست آمده (کارنو / Q-M)

طراحی مدار جمع‌کننده

بلوک دیاگرام Half Adder (جمع دو داده تک بیتی)

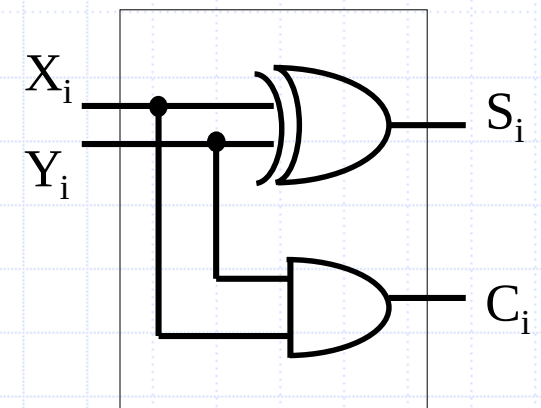


Truth Table

X_i	Y_i	C_i	S_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

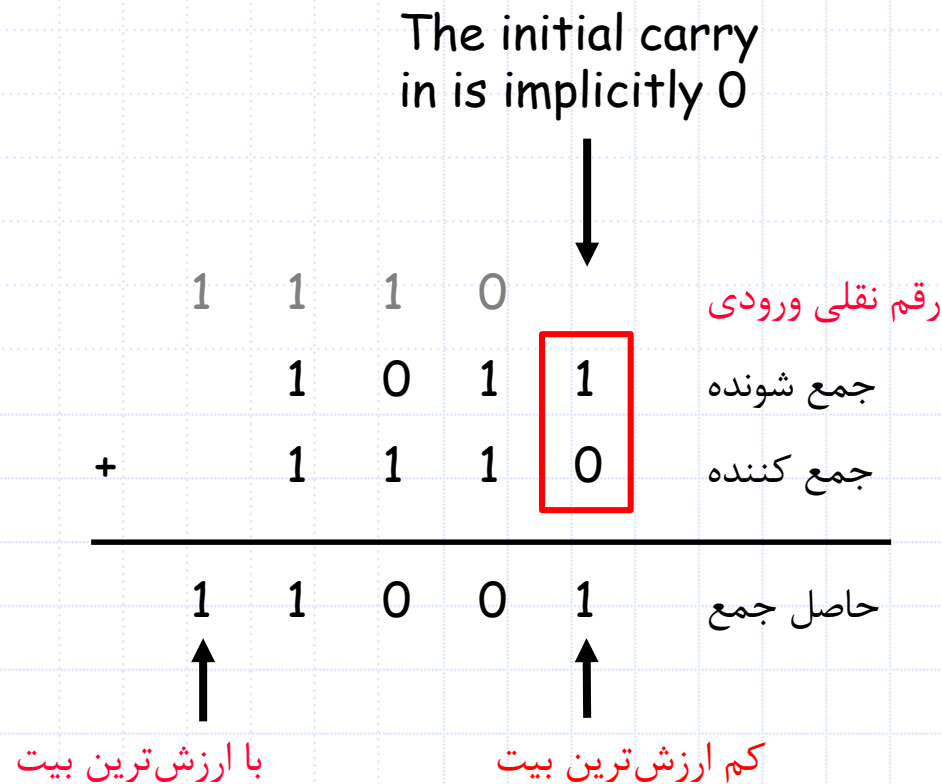


$$\begin{cases} S_i = X_i \oplus Y_i \\ C_i = X_i Y_i \end{cases}$$



طراحی مدار جمع‌کننده

در عمل داده‌ها چندبیتی هستند و در هر طبقه، رقم نقلی طبقه قبلی هم باید در نظر گرفته شود.



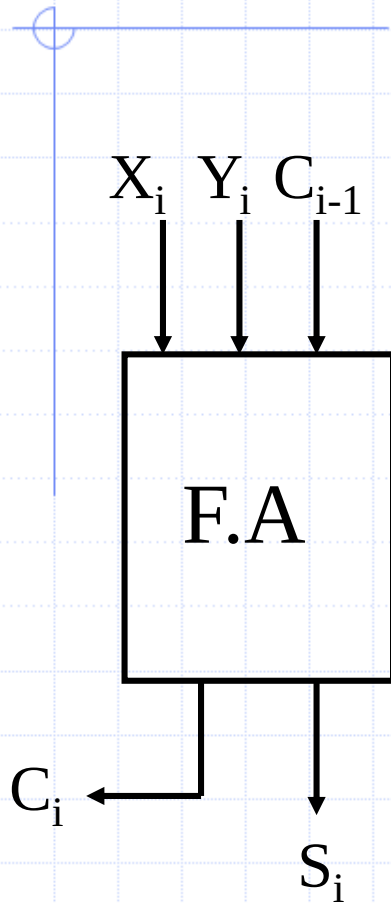
طراحی مدار جمع‌کننده

□ **Half Adder:** یک مدار ترکیبی با دو ورودی و دو خروجی است که دو بیت دودویی را با هم جمع کرده و حاصل جمع و رقم نقلی را محاسبه می‌کند.

□ **Full Adder:** یک مدار ترکیبی با سه ورودی و دو خروجی است که دو بیت داده و یک رقم نقلی را با هم جمع کرده و حاصل جمع و رقم نقلی را محاسبه می‌کند.

طراحی مدار جمع کننده

بلوک دیاگرام Full Adder



Truth Table

X_i	Y_i	C_{i-1}	C_i	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\rightarrow \begin{cases} S_i = X_i \oplus Y_i \oplus C_{i-1} \\ C_i = X_i Y_i + X_i C_{i-1} + Y_i C_{i-1} \end{cases}$$

طراحی مدار جمع‌کننده

Truth Table

X	Y	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

بلوک دیاگرام Full Adder

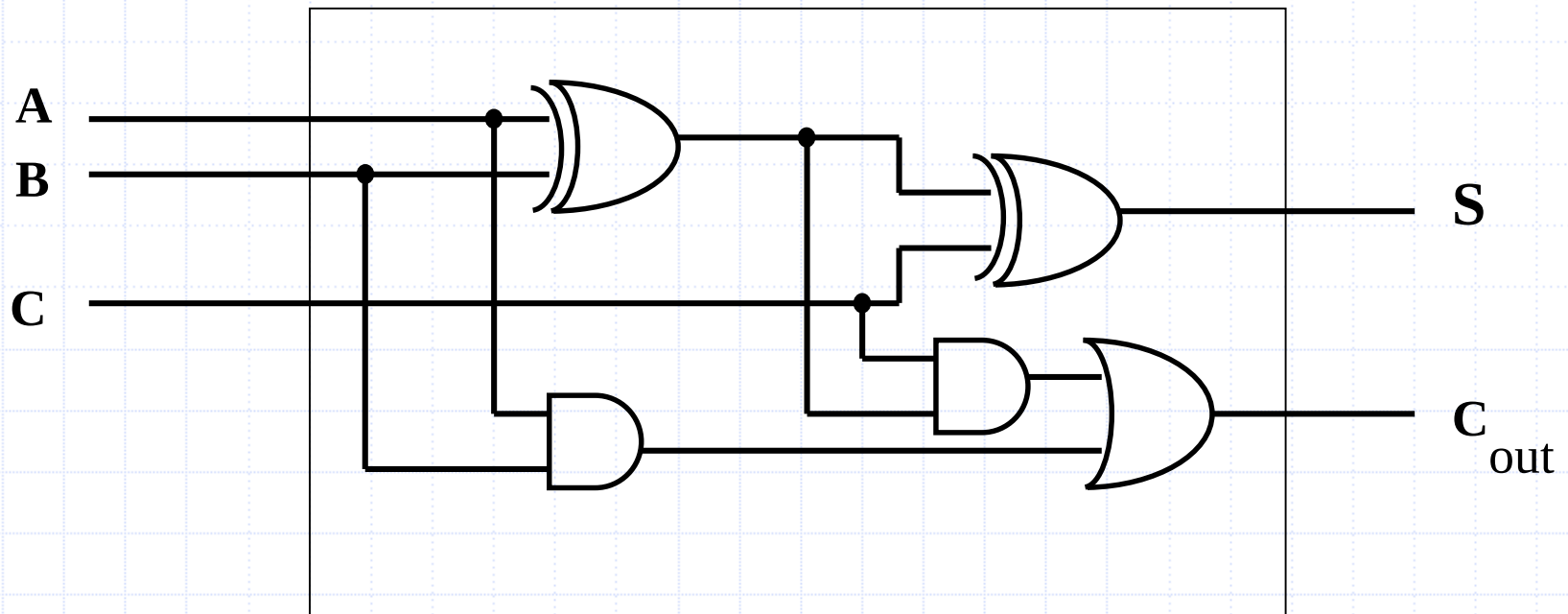
$$\begin{aligned} S &= \sum m(1,2,4,7) \\ &= X' Y' C_{in} + X' Y C_{in}' + X Y' C_{in}' + X Y C_{in} \\ &= X' (Y' C_{in} + Y C_{in}') + X (Y' C_{in}' + Y C_{in}) \\ &= X' (Y \oplus C_{in}) + X (Y \oplus C_{in})' \\ &= X \oplus Y \oplus C_{in} \end{aligned}$$

$$\begin{aligned} C_{out} &= \sum m(3,5,6,7) \\ &= X' Y C_{in} + X Y' C_{in} + X Y C_{in}' + X Y C_{in} \\ &= (X' Y + X Y') C_{in} + XY(C_{in}' + C_{in}) \\ &= (X \oplus Y) C_{in} + XY \end{aligned}$$

طراحی مدار جمع کننده

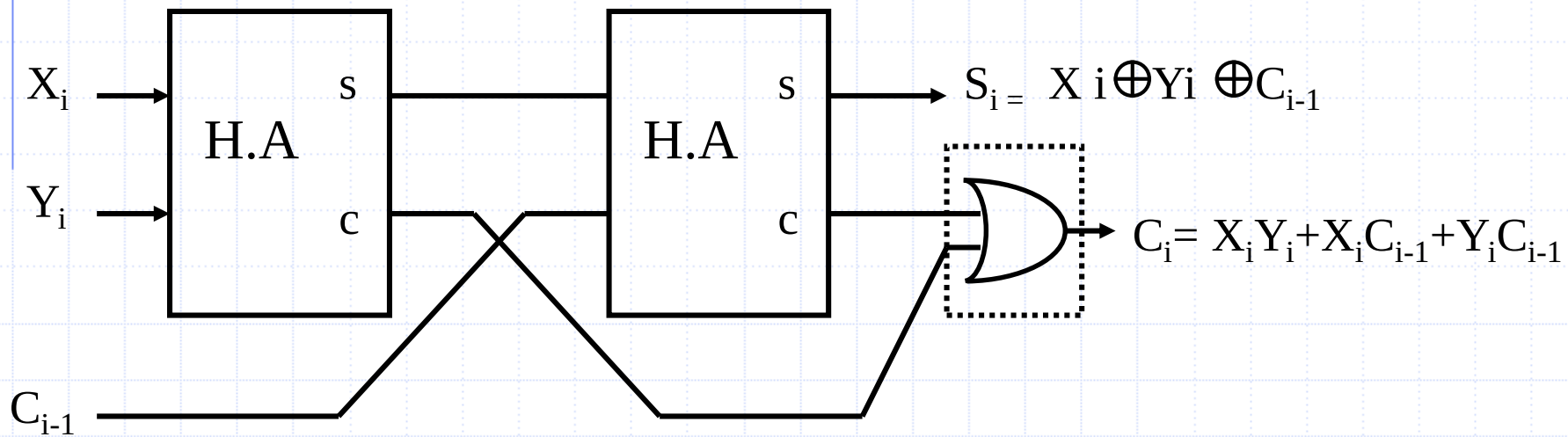
بلوک دیاگرام Full Adder

$$C_{out} = C (A \oplus B) + AB$$



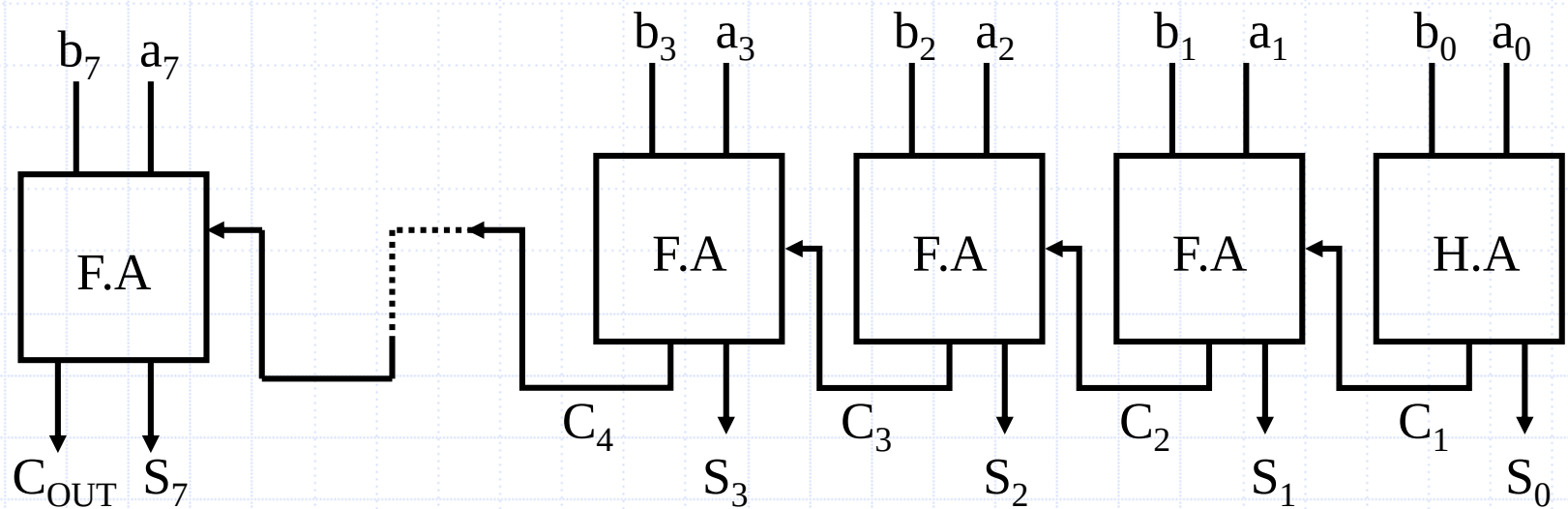
طراحی مدار جمع‌کننده

یک Full adder را می‌توان توسط ۲ عدد Half adder پیاده‌سازی کرد.



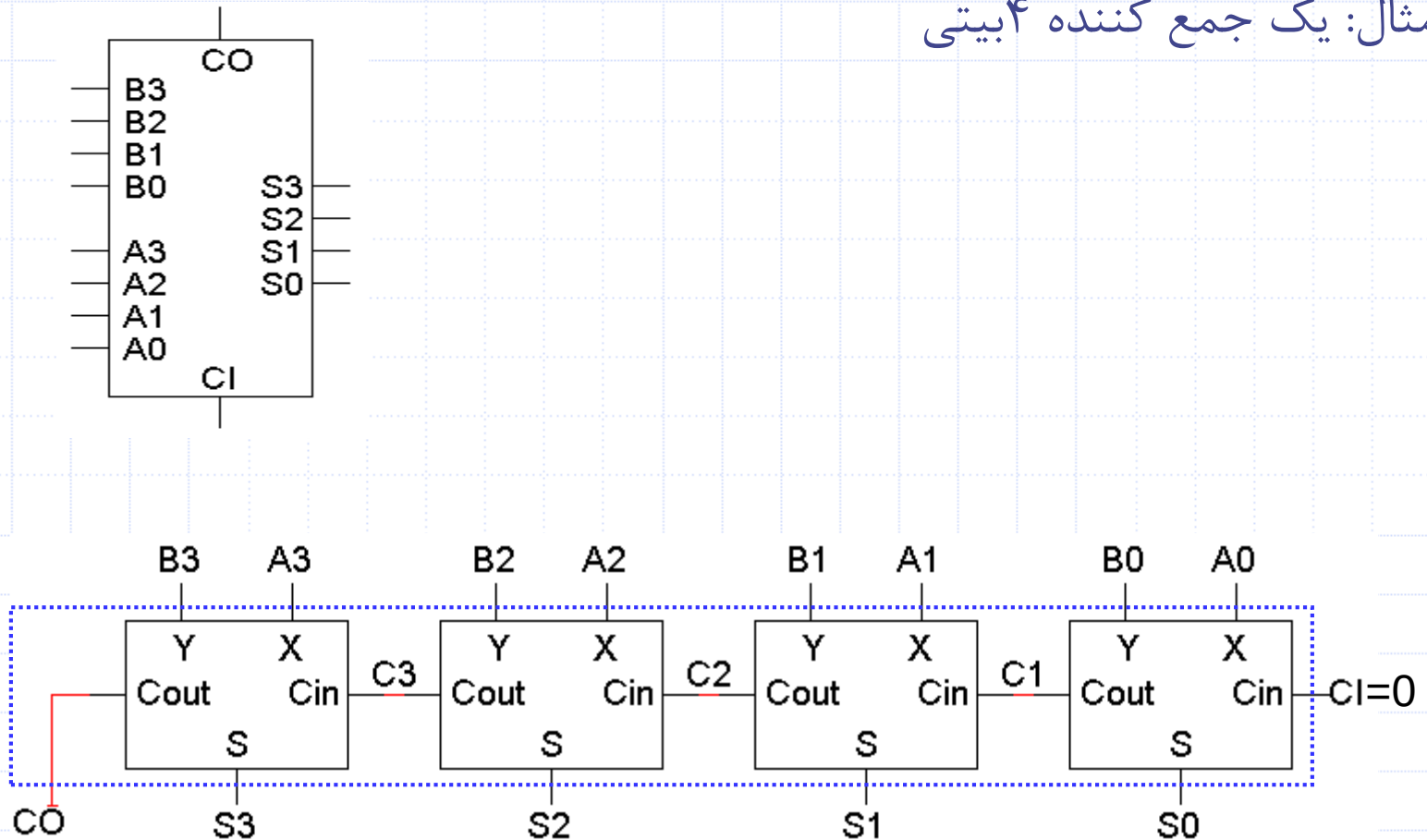
طراحی مدار جمع‌کننده چند بیتی

Ripple Carry Adder (RCA)



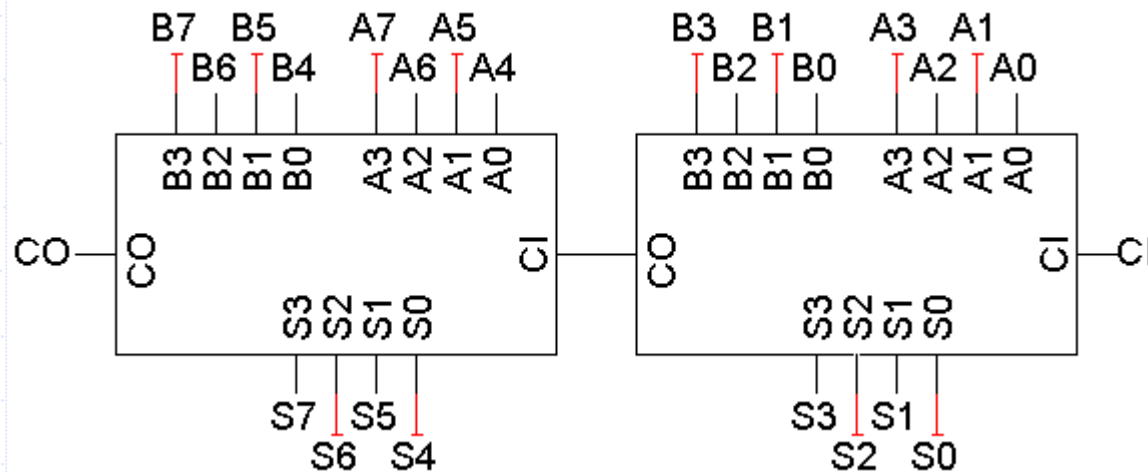
طراحی مدار جمع کننده چند بیتی

مثال: یک جمع کننده ۴ بیتی



طراحی مدار جمع‌کننده چند بیتی

مثال: ساخت یک جمع کننده ۸-بیتی با استفاده جمع کننده های ۴-بیتی

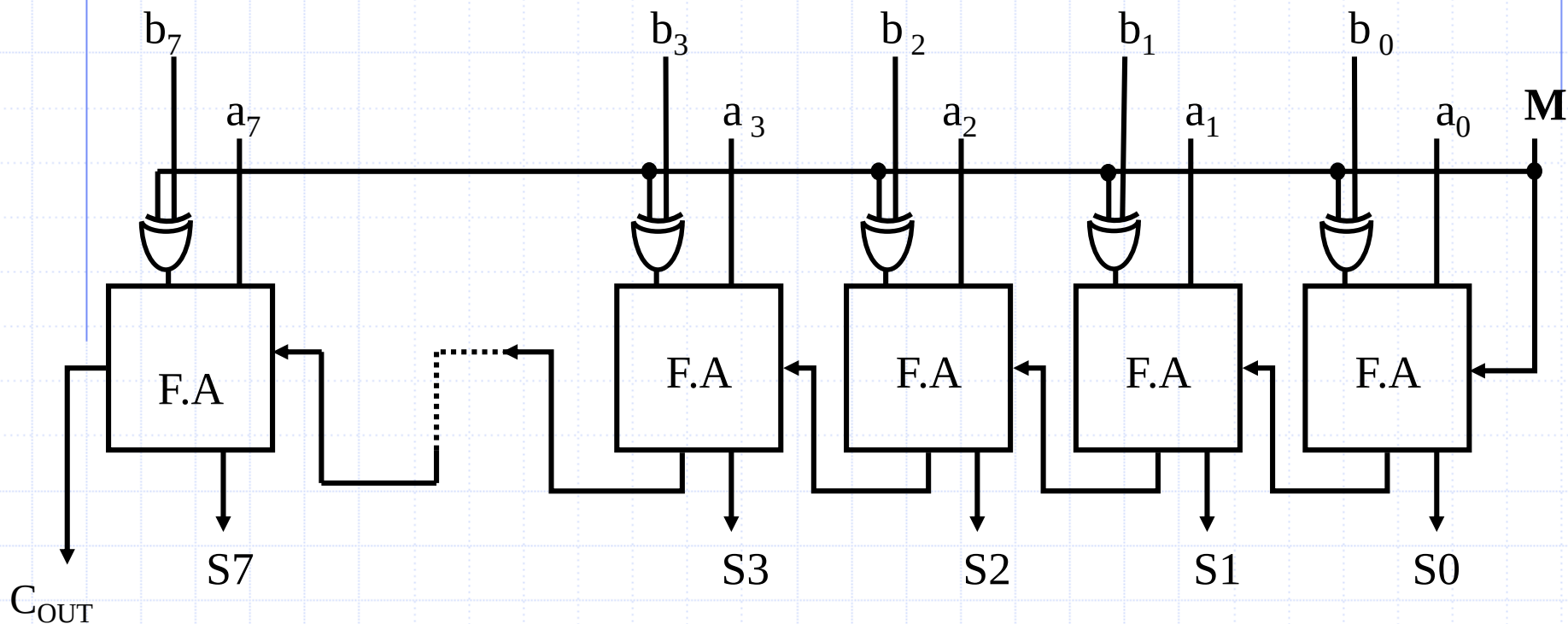


طراحی مدار تفریق‌کننده چند بیتی

- با توجه به اینکه $x - y = x + (-y) = x + y' + 1$ ، میتوان از مدار جمع‌کننده برای تفریق نیز استفاده کرد، به اینصورت که یکی از ورودیها، x و ورودی دیگر y' باشد و برای لحاظ کردن $+1$ هم، از رقم نقلی ورودی برابر با 1 در طبقه پایین استفاده کنیم.
- y' را میتوان با XOR کردن y با مقدار 1 به دست آورد: $(y \oplus 1) = y'$

طراحی مدار جمع‌کننده و تفریق‌کننده

Ripple Carry Adder (RCA)



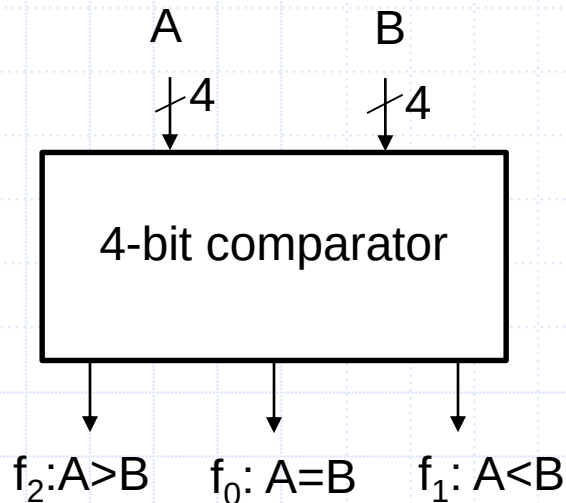
If $M = 0 \rightarrow A + B$

If $M = 1 \rightarrow A - B \text{ or } (A + \bar{B} + 1)$

طراحی مدار مقایسه‌کننده ۴ بیتی

ورودی: دو مقدار ۴ بیتی

$A = A_3A_2A_1A_0$ و $B = B_3B_2B_1B_0$



خروجی: سه بیت f_2 , f_1 , f_0 برای تعیین $A < B$, $A > B$, $A = B$ به طوری که:

if $(A=B)$ **then** $f_0=1$ **else** $f_0=0$

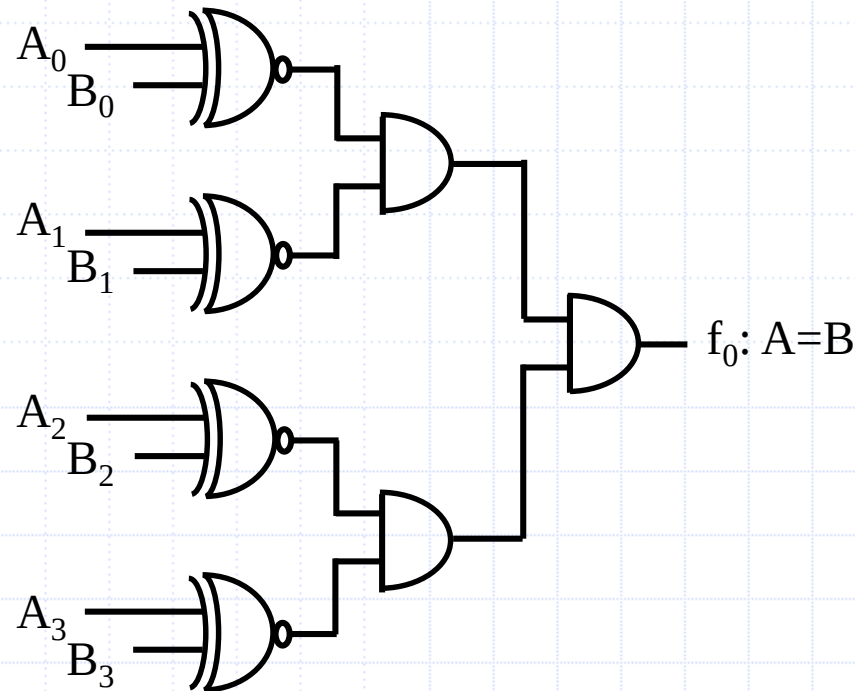
if $(A < B)$ **then** $f_1=1$ **else** $f_1=0$

if $(A > B)$ **then** $f_2=1$ **else** $f_2=0$

طراحی مدار مقایسه‌کننده ۴ بیتی

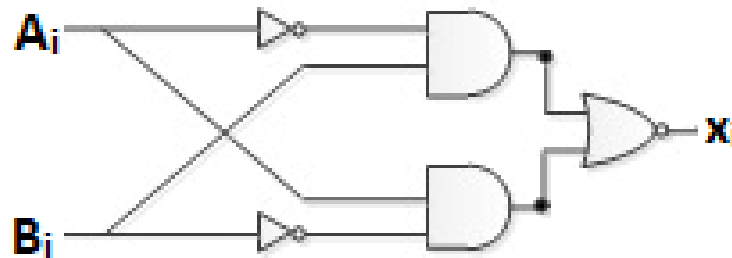
برای تشخیص تساوی:

$A=B$ if $A_3=B_3, A_2=B_2, A_1=B_1, A_0=B_0$



طراحی مدار مقایسه کننده ۴ بیتی

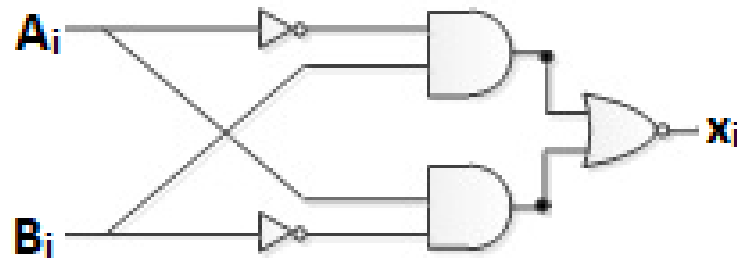
Define: $x_i = A_i \odot B_i = A_i B_i + A_i' B_i' = (A_i \oplus B_i)' = (A_i B_i' + A_i' B_i)'$



$A=B$ if $x_3 x_2 x_1 x_0 = 1$

طراحی مدار مقایسه کننده ۴ بیتی

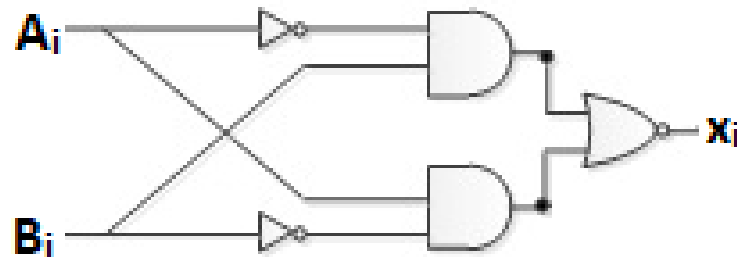
Define: $x_i = A_i \odot B_i = A_i B_i + A_i' B_i' = (A_i \oplus B_i)' = (A_i B_i' + A_i' B_i)'$



$$A > B \quad \text{if} \quad A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0' = 1$$

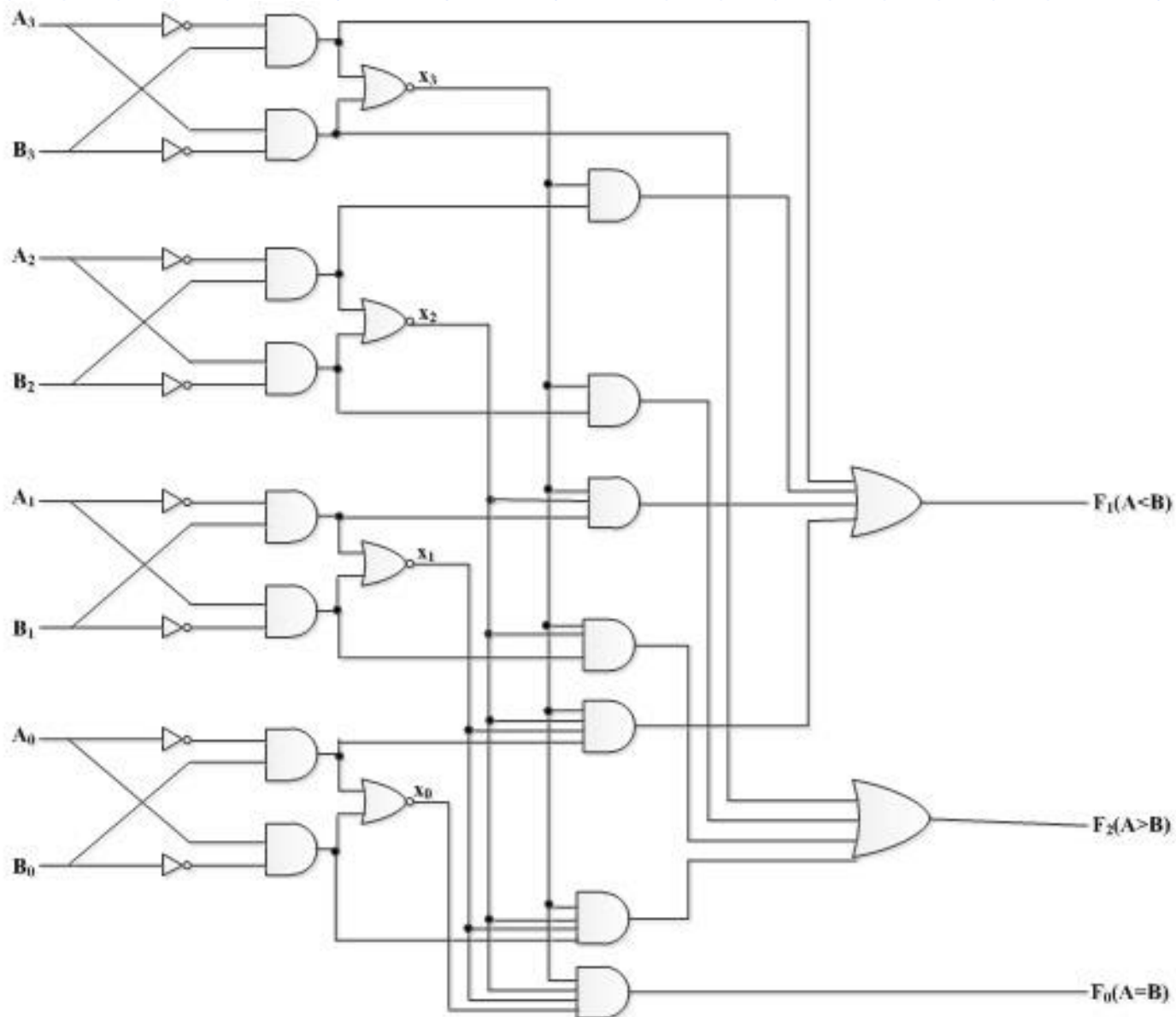
طراحی مدار مقایسه کننده ۴ بیتی

Define: $x_i = A_i \odot B_i = A_i B_i + A_i' B_i' = (A_i \oplus B_i)' = (A_i B_i' + A_i' B_i)'$

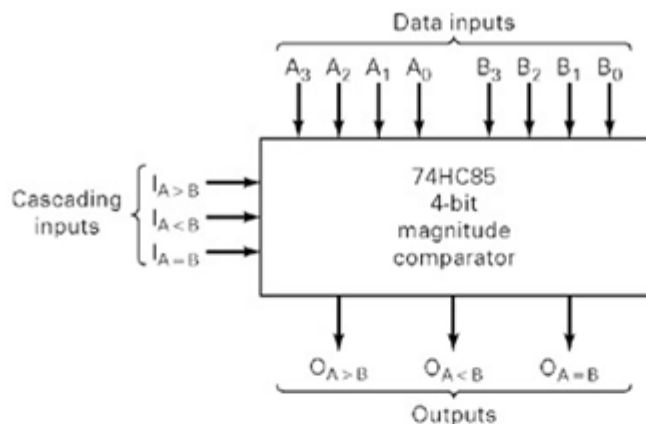


$$A < B \quad \text{if} \quad A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0 = 1$$

طراحی مدار مقایسه‌کننده ۴ بیتی



طراحی مدار مقایسه‌کننده ۴ بیتی



TRUTH TABLE

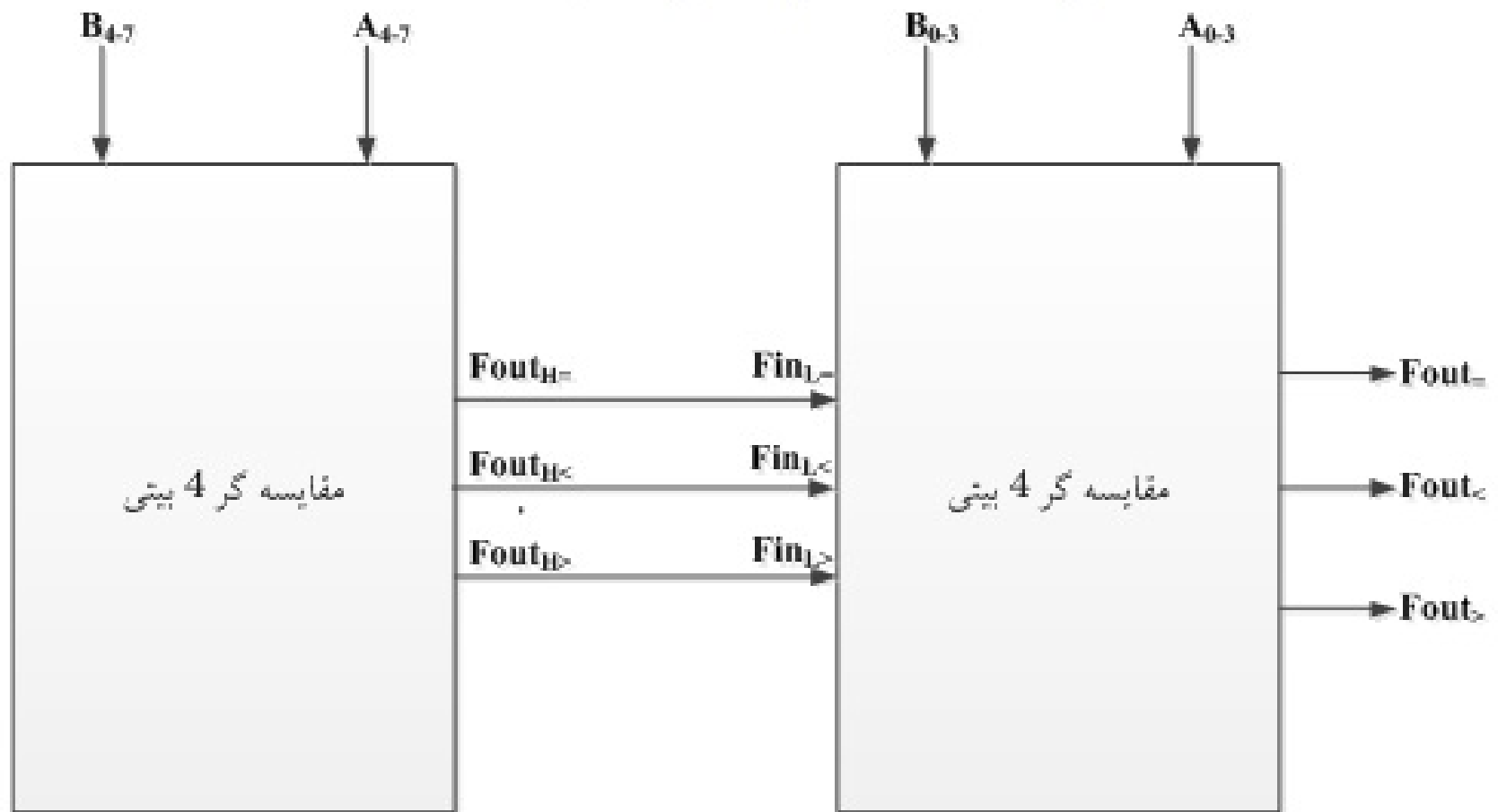
COMPARING INPUTS				CASCADING INPUTS			OUTPUTS		
A ₃ , B ₃	A ₂ , B ₂	A ₁ , B ₁	A ₀ , B ₀	I _{A>B}	I _{A<B}	I _{A=B}	O _{A>B}	O _{A<B}	O _{A=B}
A ₃ > B ₃	X	X	X	X	X	X	H	L	L
A ₃ < B ₃	X	X	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ > B ₂	X	X	X	X	X	H	L	L
A ₃ = B ₃	A ₂ < B ₂	X	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ > B ₁	X	X	X	X	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ < B ₁	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ > B ₀	X	X	X	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ < B ₀	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	H	L	L	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	L	H	L	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	X	X	H	L	L	H
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	L	L	L	H	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	H	H	L	L	L	L

H = HIGH Voltage Level

L = LOW Voltage Level

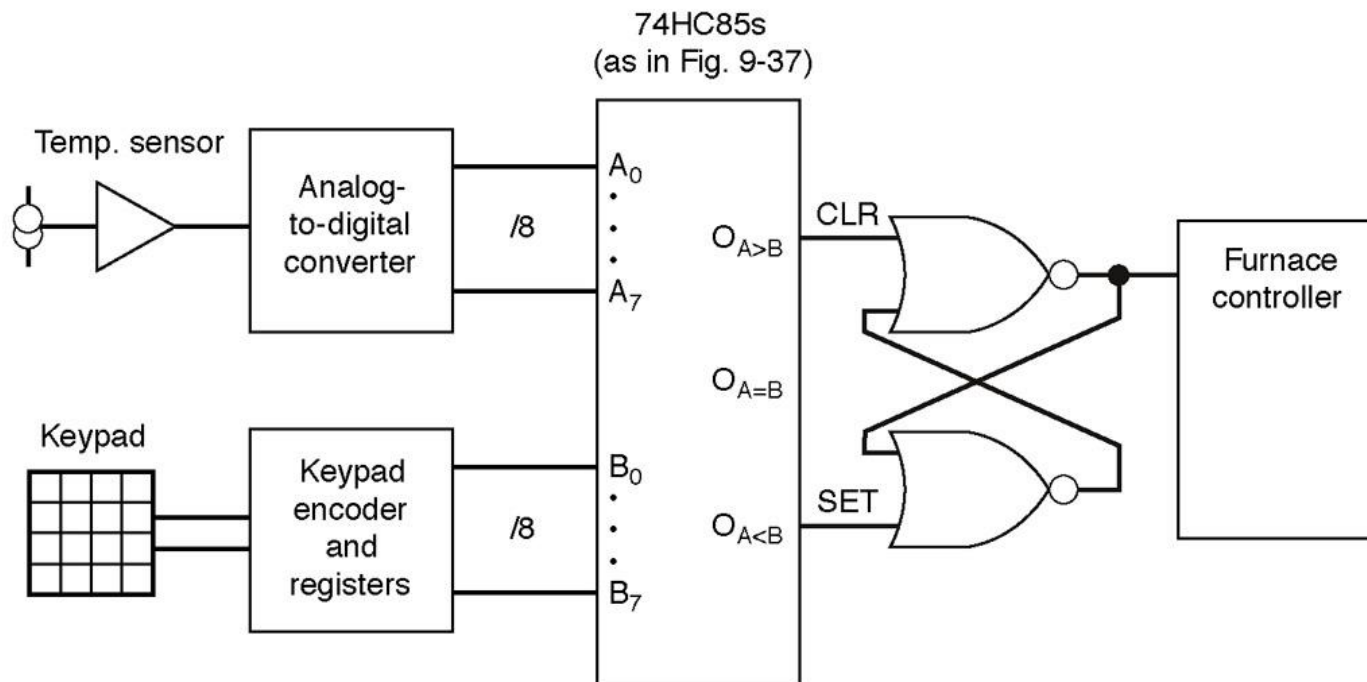
X = Immaterial

طراحی مدار مقایسه کننده ۸ بیتی



کاربرد مدار مقایسه‌کننده

مثال: مدار یک کنترلر ترموستات



طراحی مدار بیت توازن

Truth table

a	b	c	Even Parity
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

مثال: مدار تولید بیت توازن زوج برای داده 3 بیتی

$$F(a,b,c) = \sum m(1,2,4,7)$$

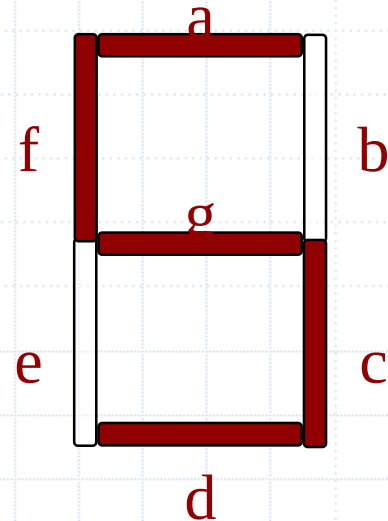
		b c			
a		00	01	11	10
	0	0	1	0	1
	1	1	0	1	0

$$F(a,b,c) = ab'c' + a'b'c + abc + a'bc'$$

$$F(a,b,c) = (a \oplus b) \oplus c$$

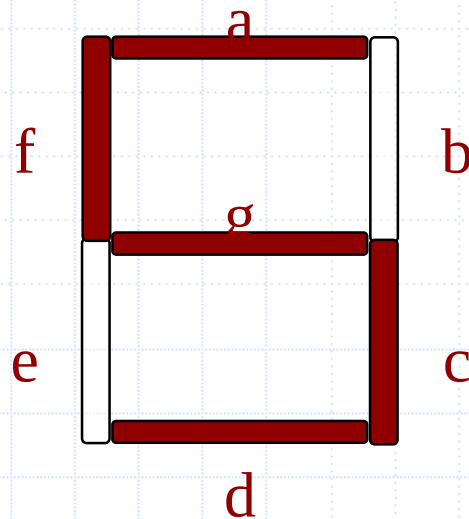
طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

	A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
.	.	.	.	.
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1



طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

0	a,b,c,d,e,f
1	b,c
2	a,b,d,e,g
3	a,b,c,d,g
4	b,c,f,g
5	a,c,d,f,g
6	a,c,d,e,f,g
7	a,b,c
8	a,b,c,d,e,f,g
9	a,b,c,d,f,g



طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

Dec	Inputs				Outputs					
	w	x	y	z	a	b	c	d	e	.
0	0	0	0	0	1	1	1	1	1	.
1	0	0	0	1	0	1	1	0	0	.
2	0	0	1	0	1	1	0	1	1	.
...	...	...	...	...	...	...	...	...	...	.
7	0	1	1	1	1	1	1	0	0	.
8	1	0	0	0	1	1	1	1	1	.
9	1	0	0	1	1	1	1	1	0	.

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

		yz			
wx		00	01	11	10
	00	1	0	1	1
	01	0	1	1	1
	11				
	10	1	1		

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

wx \ yz	yz			
	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

Put in “X” (don’t care), and interpret as either 1 or 0 as desired

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

		yz			
		00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$F_{a1} = y$$

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

		yz			
		00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$F_{a2} = w$$

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

		yz			
		00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$F_{a3} = \overline{x}z$$

		yz			
		00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$F_{a4} = xz$$

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “a” :

		yz			
		00	01	11	10
wx	00	1	0	1	1
	01	0	1	1	1
	11	X	X	X	X
	10	1	1	X	X

$$F = y + w + \overline{xz} + xz$$

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

Dec	Inputs				Outputs					
	w	x	y	z	a	b	c	d	e	.
0	0	0	0	0	1	1	1	1	1	.
1	0	0	0	1	0	1	1	0	0	.
2	0	0	1	0	1	1	0	1	1	.
...	...	...	...	...	...	...	...	...	...	.
7	0	1	1	1	1	1	1	0	0	.
8	1	0	0	0	1	1	1	1	1	.
9	1	0	0	1	1	1	1	1	0	.

طراحی مدار مبدل کد BCD به نمایشگر ۷ قسمتی

For segment “b” :

		yz			
		00	01	11	10
wx	00	1	1	1	1
	01	1	0	1	0
	11				
	10	1	1		

See if you can complete this example.

رمزگشا – دیکدر (Decoder)

✓ دیکدر n به 2^n یک مدار ترکیبی است با n خط ورودی و 2^n سیگنال خروجی.

✓ در حال کار، بسته به ترکیب ورودیها، فقط یکی از خروجیها فعال خواهد بود. وقتی

ورودی 00..00 است، خروجی O_0 فعال خواهد بود، وقتی ورودیها 00..01 است،

خروجی O_1 فعال خواهد بود و ...

✓ می توان دید که دیکدر فعال بالا (Active High) در واقع مینترم ها را می سازد و

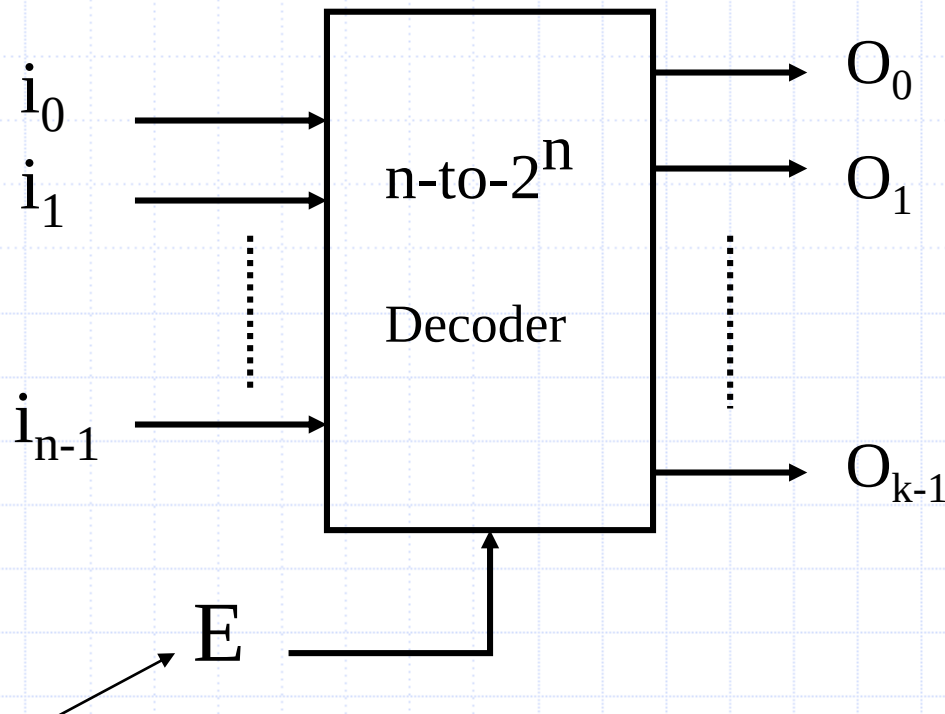
دیکدر فعال پایین (Active Low) ماکسترمها را.

رمزگشا – دیکدر (Decoder)

✓ ماجول دیکدر n به 2^n ($k = 2^n$)

✓ وقتی ورودی $00..00$ است، خروجی O_0 فعال خواهد بود، وقتی ورودیها $00..01$ است، خروجی O_1 فعال خواهد بود و ...

✓ در واقع خروجیها معادل با مینترمها یا ماکسترمها هستند. خروجی O_0 معادل با m_0 یا M_0 و ...



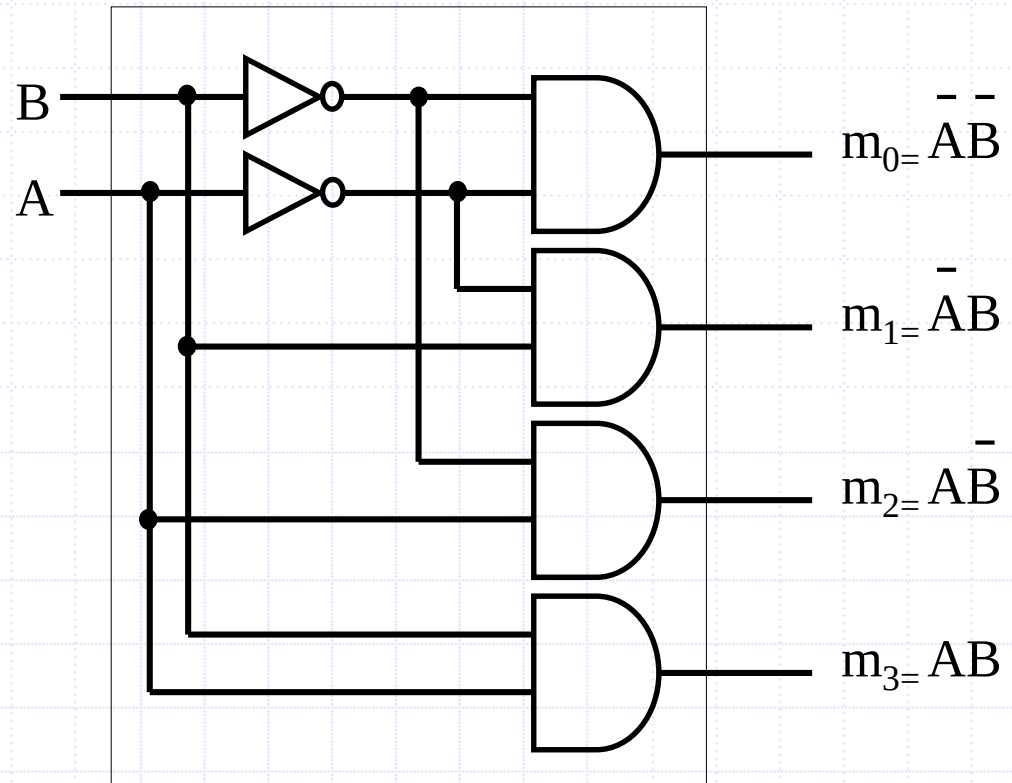
معمولا Active Low هستند.

رمزگشا – دیکدر (Decoder)

✓ دیاگرام منطقی (موازی و خروجی های فعال بالا)
خروجیها معادل با مینترمها هستند.

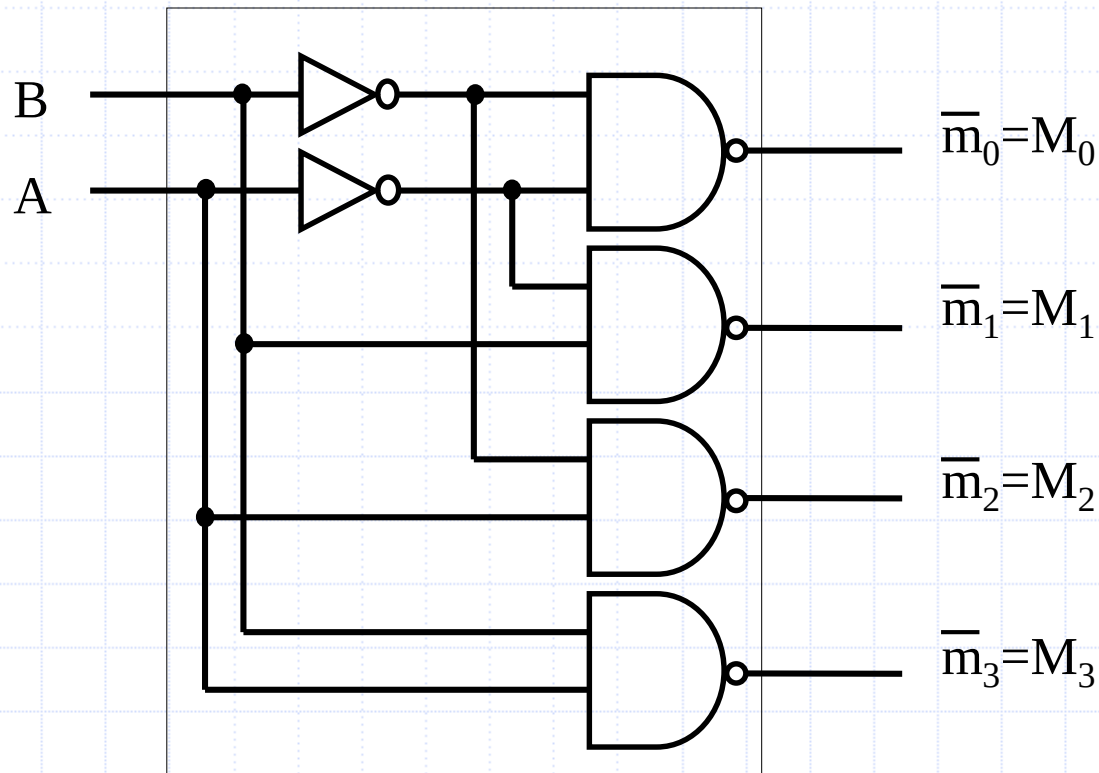
Truth Table

E	A	B	m_0	m_1	m_2	m_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	×	×	0	0	0	0



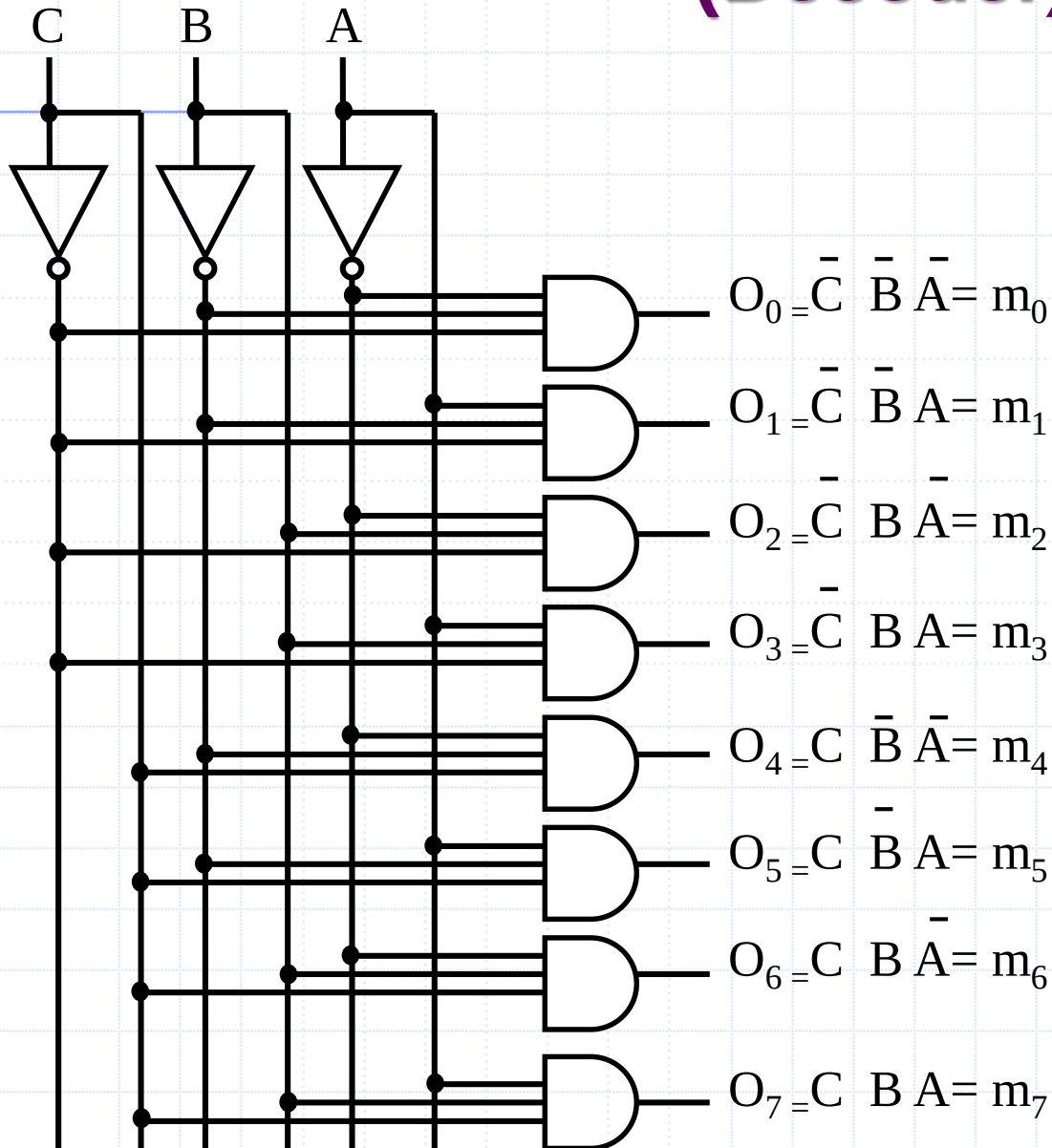
رمزگشا – دیکدر (Decoder)

✓ دیاگرام منطقی (موازی و خروجی های فعال پائین)
خروجیها معادل با ماکسترهما هستند.



رمزگشا – دیکدر (Decoder)

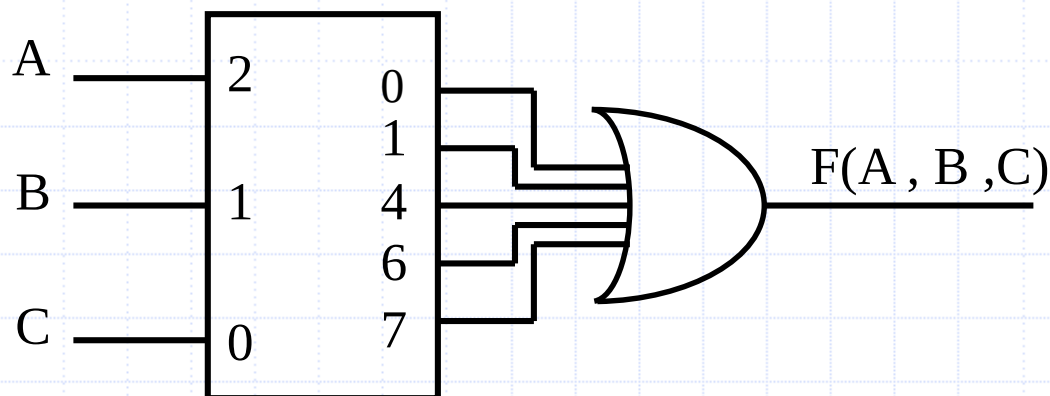
✓ دیکدر ۳ به ۲³



پیاده سازی توابع منطقی با استفاده از دیکدرها

مثال:

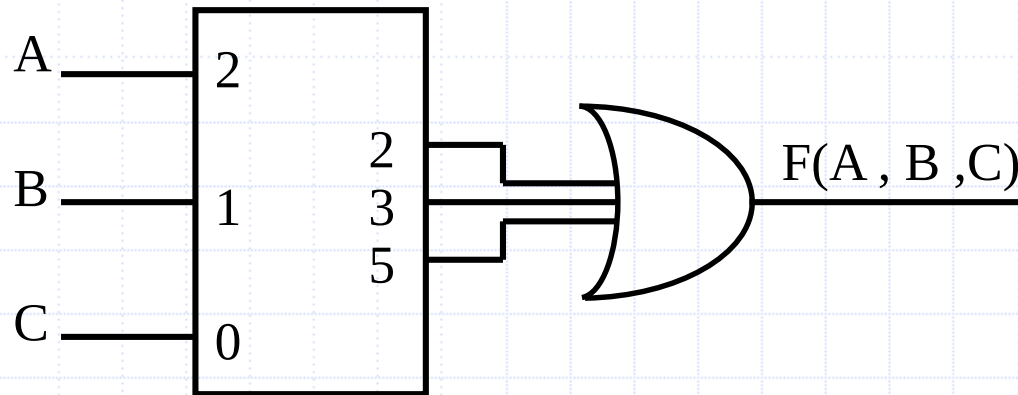
$$F(A, B, C) = \sum m(0, 1, 4, 6, 7) = \prod M(2, 3, 5)$$
$$= m_0 + m_1 + m_4 + m_6 + m_7$$



پیاده سازی توابع منطقی با استفاده از دیگرها

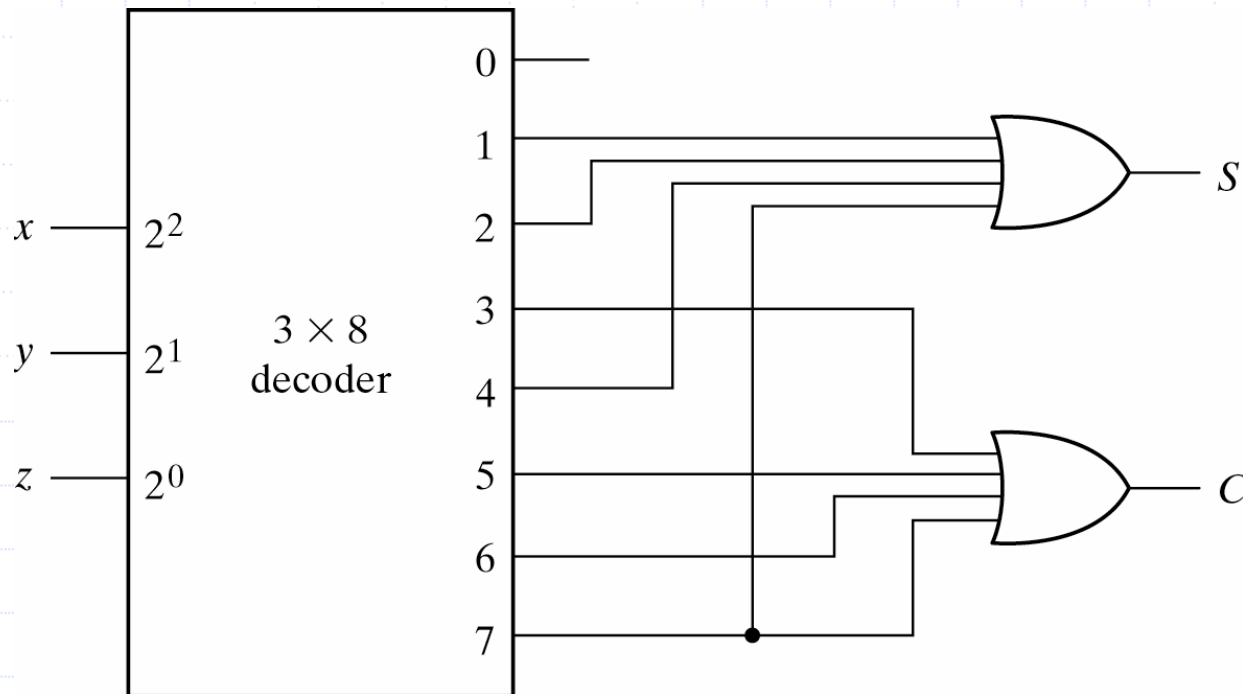
مثال:

$$F(A, B, C) = \overline{\sum m(0, 1, 4, 6, 7)} = \overline{\prod M(2, 3, 5)}$$
$$= m_2 + m_3 + m_5$$



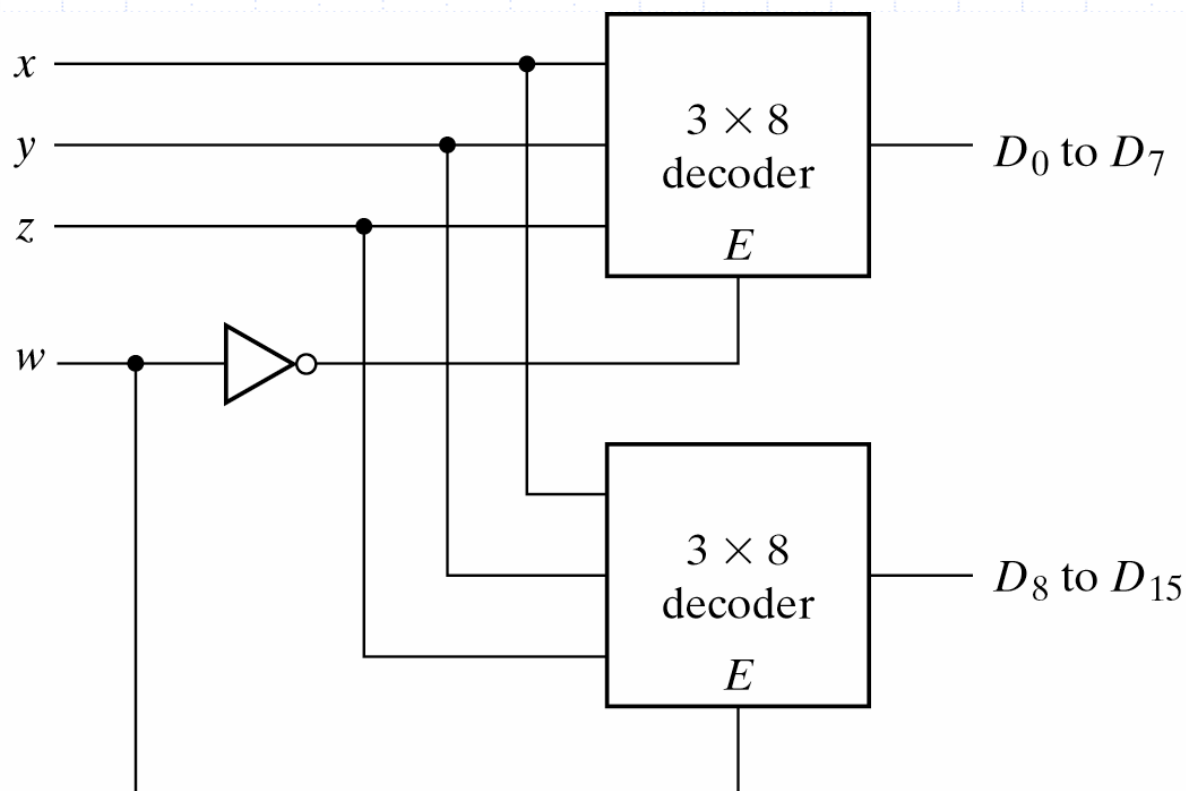
پیاده سازی توابع منطقی با استفاده از دیکدرها

مثال: ساخت یک Full adder با استفاده از دیکدر



پیاده سازی توابع منطقی با استفاده از دیکدرها

مثال: ساخت یک دیکدر ۴ به 2^4 با استفاده از دو دیکدر ۳ به 2^3



رمزگذار – انکدر (Encoder)

✓ انکدر یک ماجول ترکیبی است که برای هر سیگنال ورودی یک کد خروجی منحصر به فرد را اختصاص می دهد.

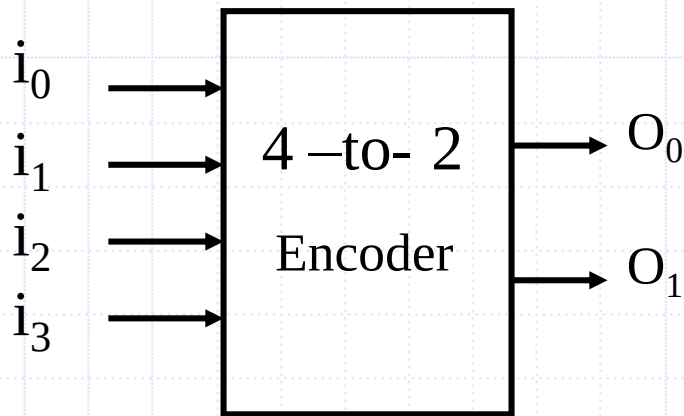
✓ عملکرد انکدر بر عکس دیکدر است.

✓ اگر یک ماجول انکدر n ورودی داشته باشد خروجی S باید در رابطه زیر صدق کند:

$$\text{or} \quad 2^s \geq n$$
$$s \geq \log_2 n$$

رمزگذار – انکدر (Encoder)

✓ مثال: انکدر با چهار خط ورودی:



i_3	i_2	i_1	i_0	O_1	O_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

رمزگذار – انکدر (Encoder)

O_1 / $i_3 i_2$ / $i_1 i_0$

x	1	x	1
0	x	x	x
x	x	x	x
0	x	x	x

$O_1 = i_2 + i_3$

O_0 / $i_3 i_2$ / $i_1 i_0$

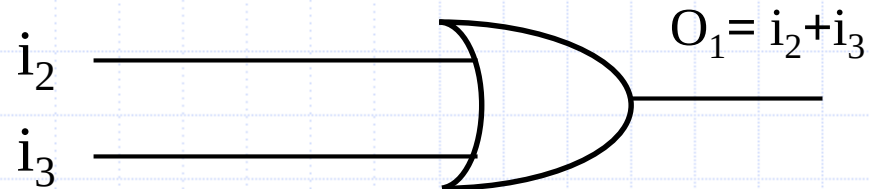
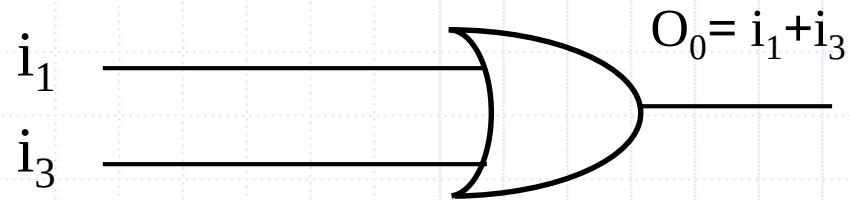
x	0	x	1
0	x	x	x
x	x	x	x
1	x	x	x

$O_0 = i_1 + i_3$

i_3	i_2	i_1	i_0	O_1	O_0	V
0	0	0	0	x	x	0
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	0	1	1	x	x	0
0	1	0	0	1	0	1
0	1	0	1	x	x	0
0	1	1	0	x	x	0
0	1	1	1	x	x	0
1	0	0	0	1	1	1
1	0	0	1	x	x	0
1	0	1	0	x	x	0
1	0	1	1	x	x	0
1	1	0	0	x	x	0
1	1	0	1	x	x	0
1	1	1	0	x	x	0
1	1	1	1	x	x	0

رمزگذار – انکدر (Encoder)

✓ مدار یک انکدر با چهار خط ورودی:



رمزگذار اولویت دار (Priority Encoder)

- ✓ انکدر اولویت دار اجازه می دهد تا چندین خط ورودی فعال شوند ولی عدد دودویی خارج شده بیانگر اندیسی است که در خطوط ورودی بالاترین اولویت را دارد.
- ✓ برای ساده کردن طراحی، بالاترین اولویت به بالاترین اندیس اختصاص یافته است و بالاترین اولویت بعدی به دومین اندیس بالاتر و الی آخر تخصیص داده شده است.

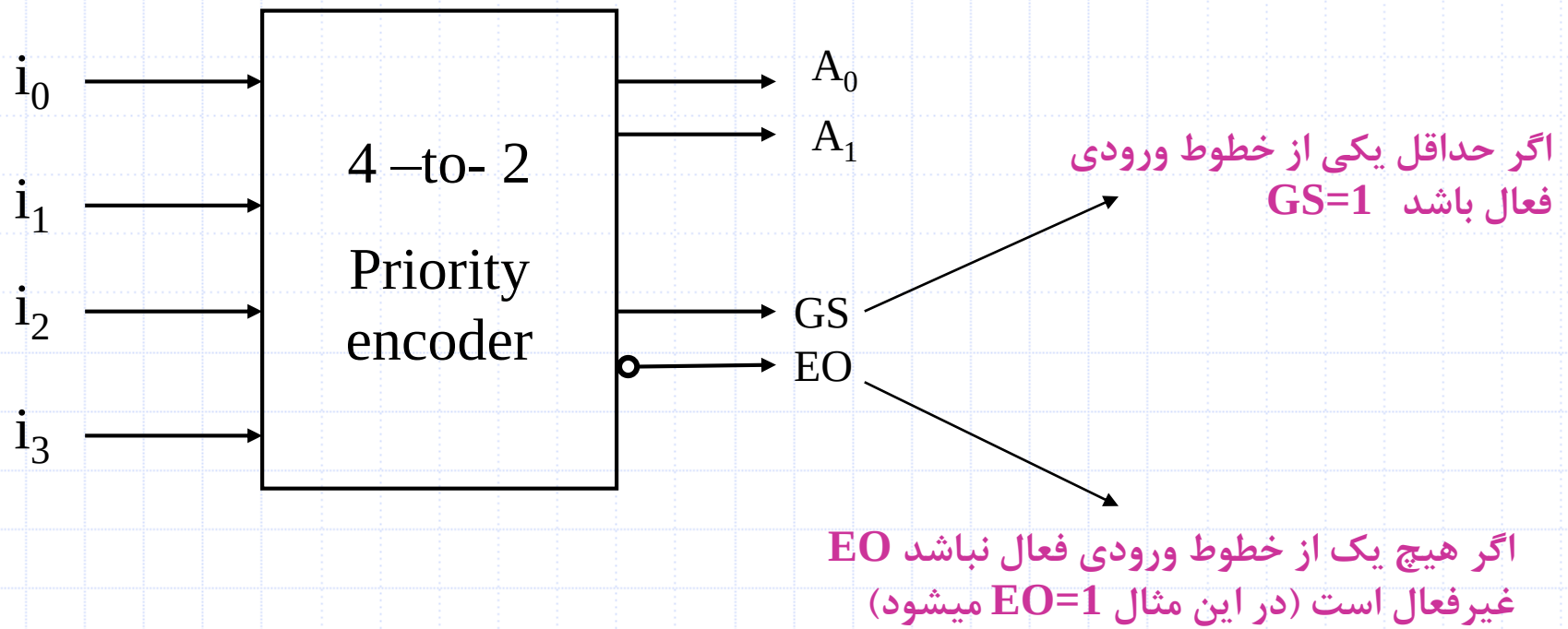
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

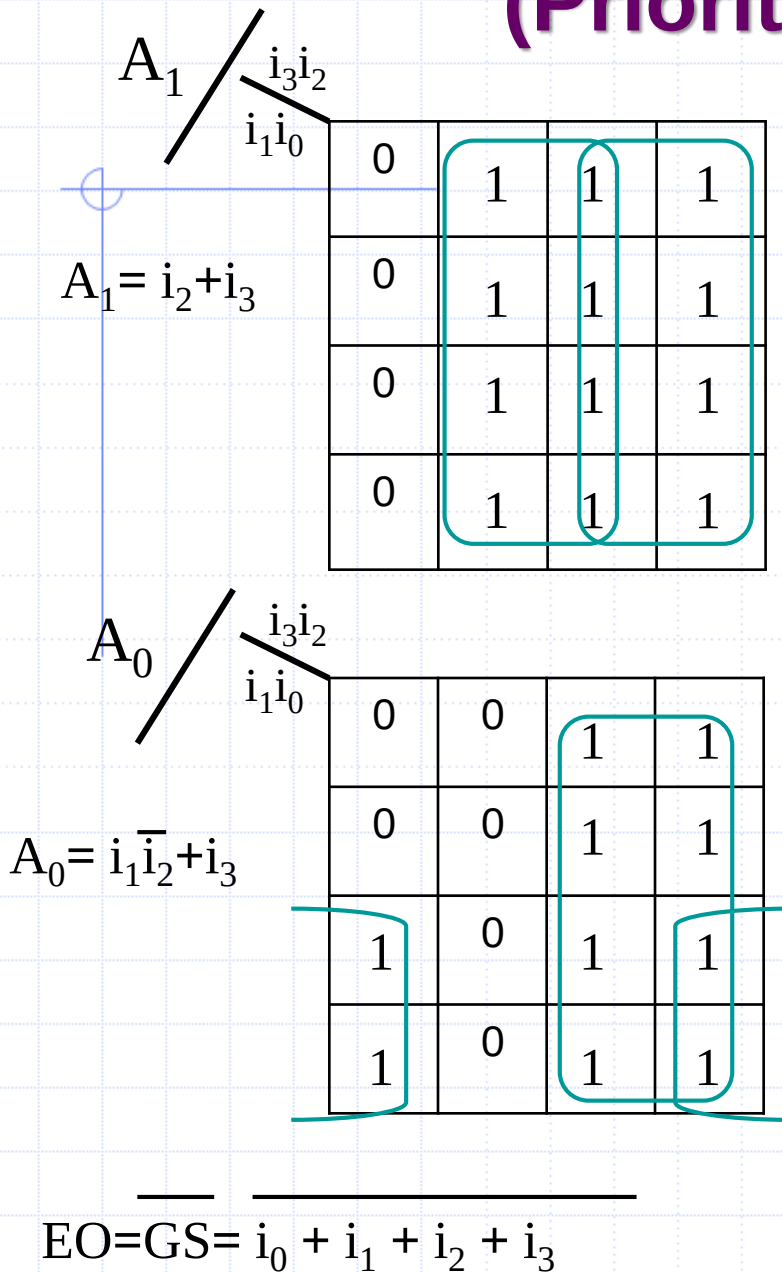
✓ مثال:

رمزگذار اولویت دار (Priority Encoder)

✓ بلوک دیاگرام یک انکدر اولویت دار با چهار خط ورودی:

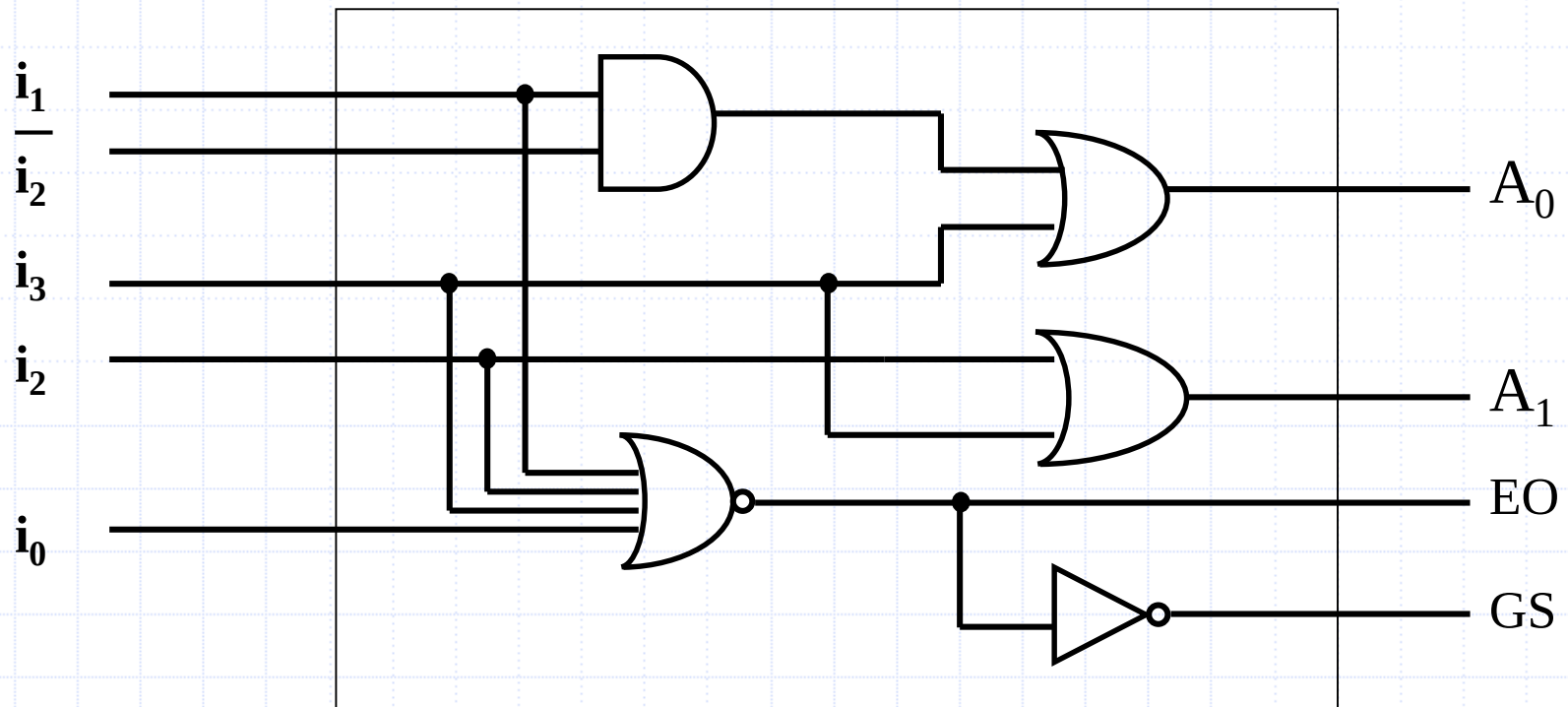


رمزگذار اولویت دار (Priority Encoder)



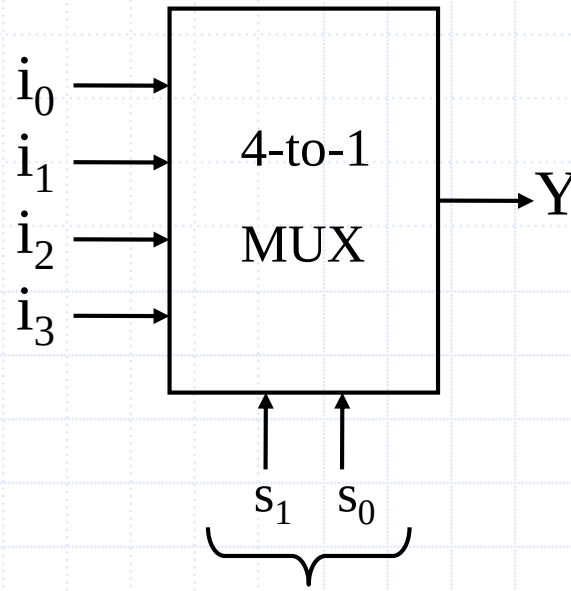
i_3	i_2	i_1	i_0	A_1	A_0	GS	EO
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	1	1	0
0	0	1	1	0	1	1	0
0	1	0	0	1	0	1	0
0	1	0	1	1	0	1	0
0	1	1	0	1	0	1	0
0	1	1	1	1	0	1	0
1	0	0	0	1	1	1	0
1	0	0	1	1	1	1	0
1	0	1	0	1	1	1	0
1	0	1	1	1	1	1	0
1	1	0	0	1	1	1	0
1	1	0	1	1	1	1	0
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	0

رمزگذار اولویت دار (Priority Encoder)



مالتی پلکسر (Multiplexer)

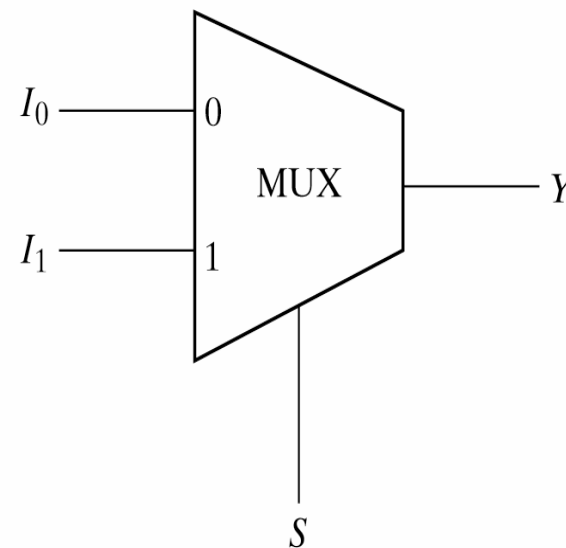
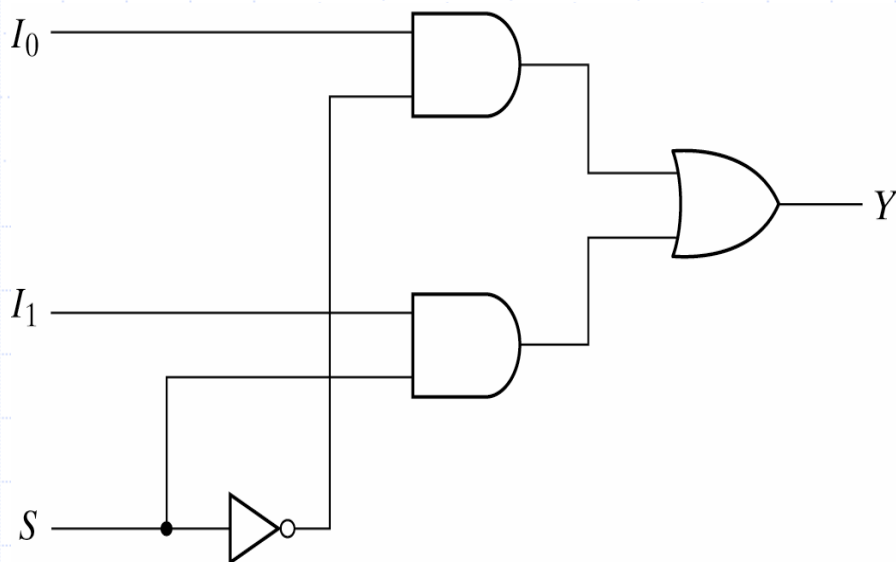
✓ به طور کلی مالتی پلکسر (انتخابگر داده) یک ماجول است که یکی از چند خط ورودی را انتخاب و آن را روی خط خروجی ظاهر می سازد.



کد انتخاب

مالتی پلکسر (Multiplexer)

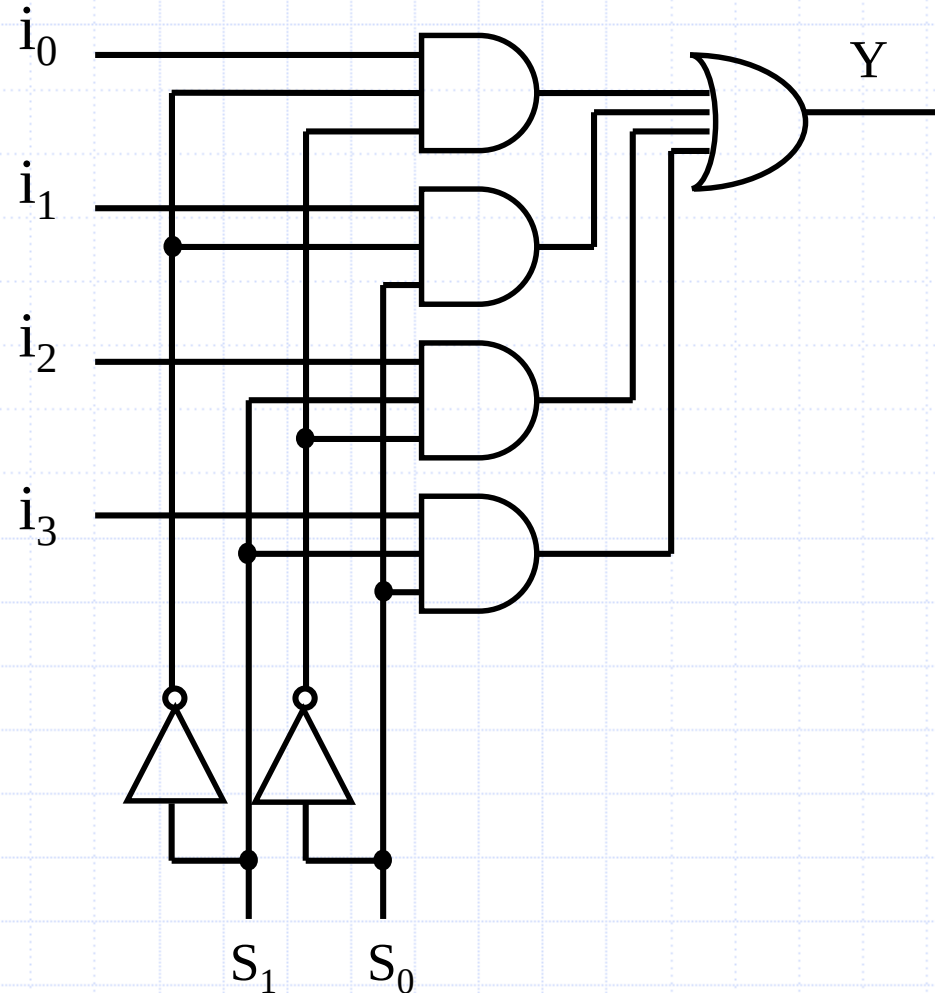
- بلوک دیاگرام و مدار یک مالتی پلکسر ۲ به ۱
- منظور از S همان ورودی انتخاب است. I_1 و I_2 نیز دو خط ورودی هستند.



مالتی پلکسر (Multiplexer)

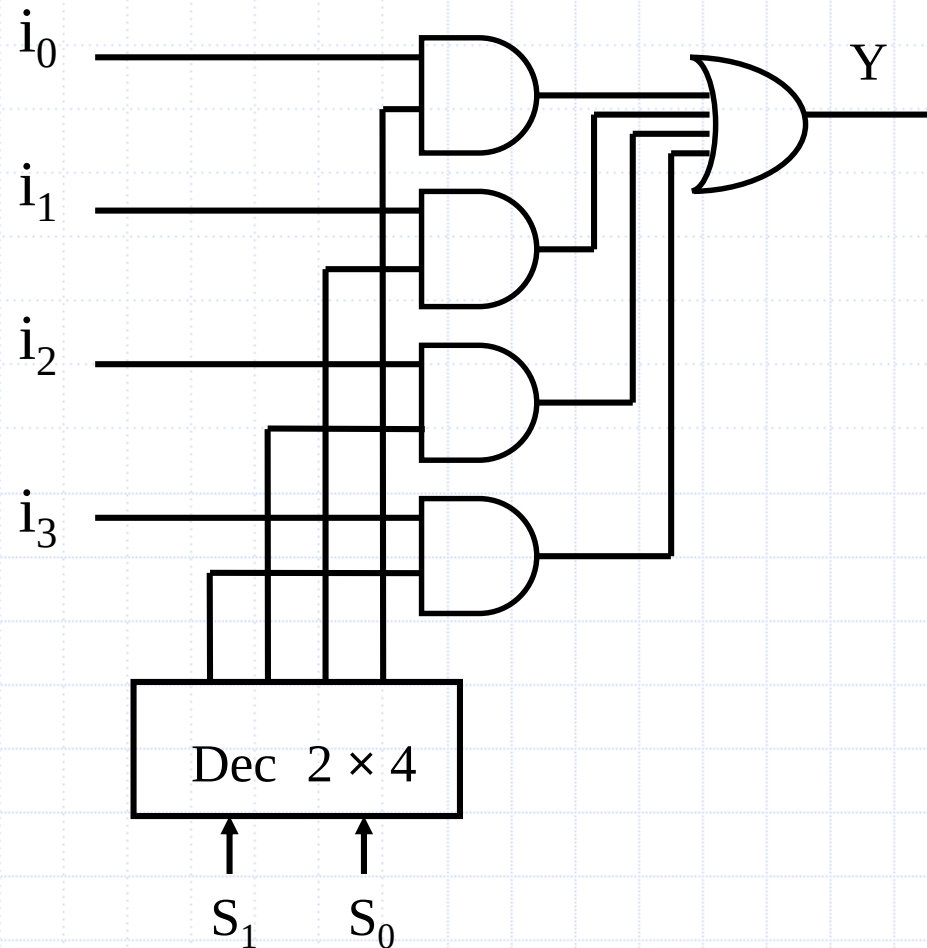
✓ مدار یک مالتی پلکسر با چهار خط ورودی و دو انتخاب کننده:

S_1	S_0	Y
0	0	i_0
0	1	i_1
1	0	i_2
1	1	i_3



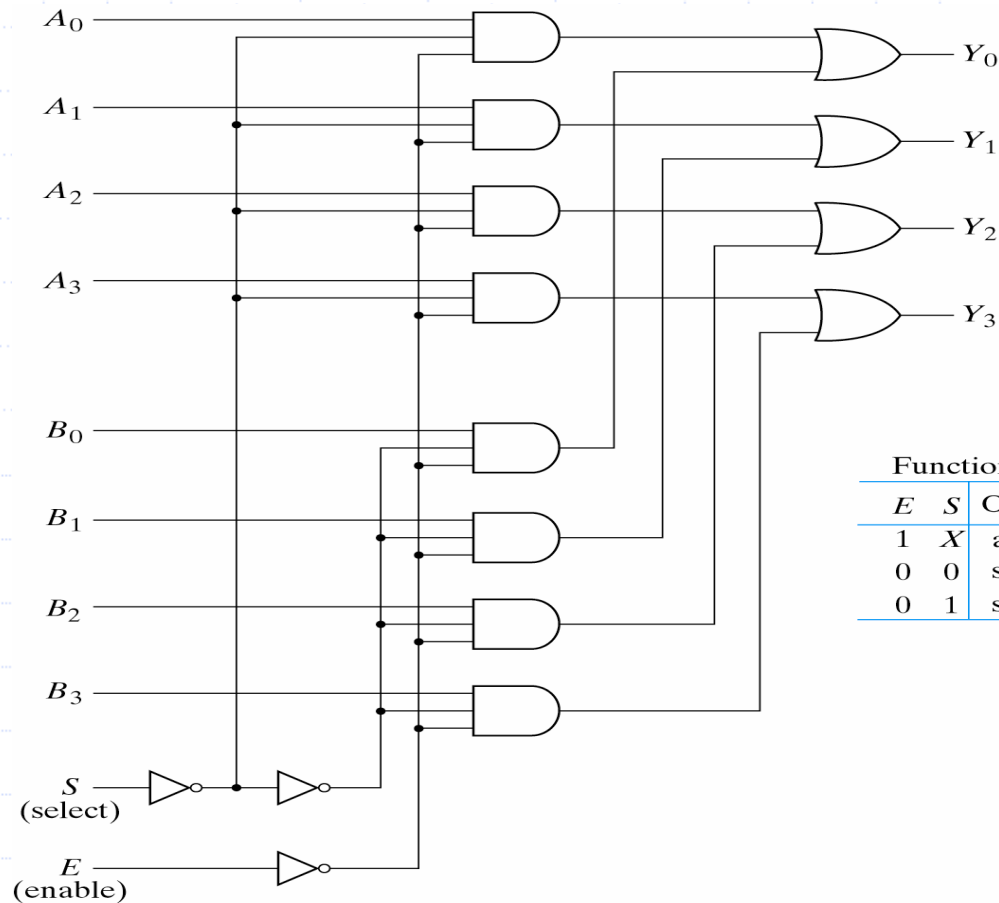
مالتی پلکسر (Multiplexer)

✓ مدار یک مالتی پلکسر با استفاده از دیگر:



مالتی پلکسر (Multiplexer)

MUX چهاربیتی ۲ به ۱



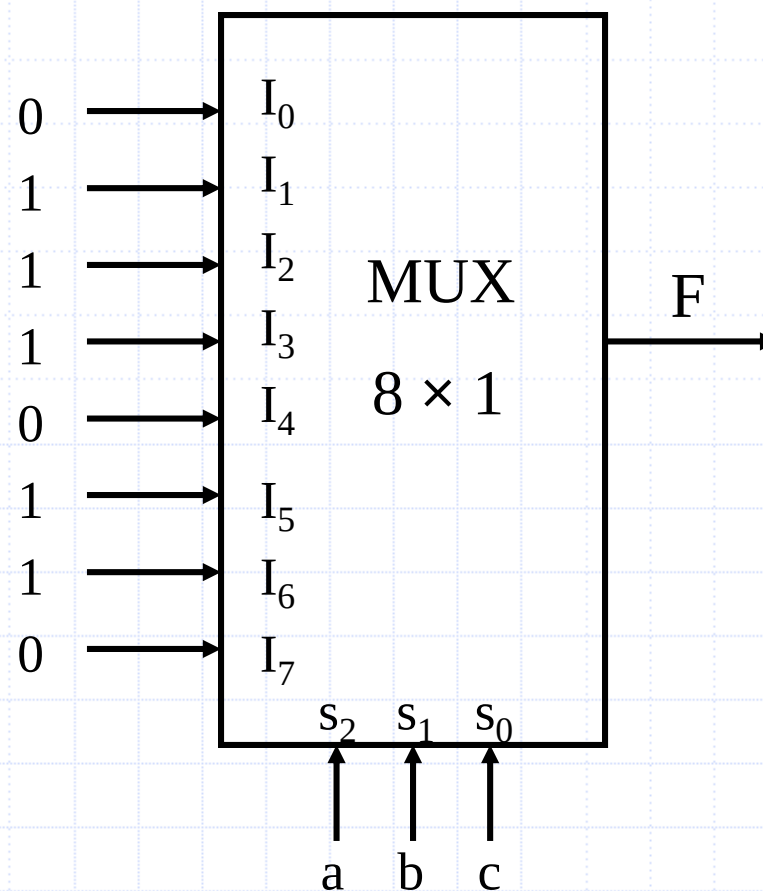
Function table		
E	S	Output Y
1	X	all 0's
0	0	select A
0	1	select B

پیاده سازی توابع با استفاده از مالتی پلکسر

$$F(A, B, C) = \sum m(1, 2, 3, 5, 6)$$

✓ مثال:

a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



پیاده سازی توابع با استفاده از مالتی پلکسر

- برای پیاده سازی تابع بولی n متغیره با مالتی پلکسر دارای $n-1$ خط انتخاب، $n-1$ تا از متغیرها را به خطوط انتخاب MUX وصل و متغیر باقیمانده را در خطوط ورودی استفاده کرد.
- اگر A متغیر منفرد (باقی مانده) تابع بولی باشد، ورودی های MUX می توانند A ، A' ، 1 و 0 باشند.
- ساده تر اینست که متغیر با پایینترین یا بالاترین ارزش را بعنوان متغیر منفرد انتخاب کرده، بقیه متغیرها را به ورودی های انتخاب MUX بدهیم.

پیاده سازی توابع با استفاده از مالتی پلکسر

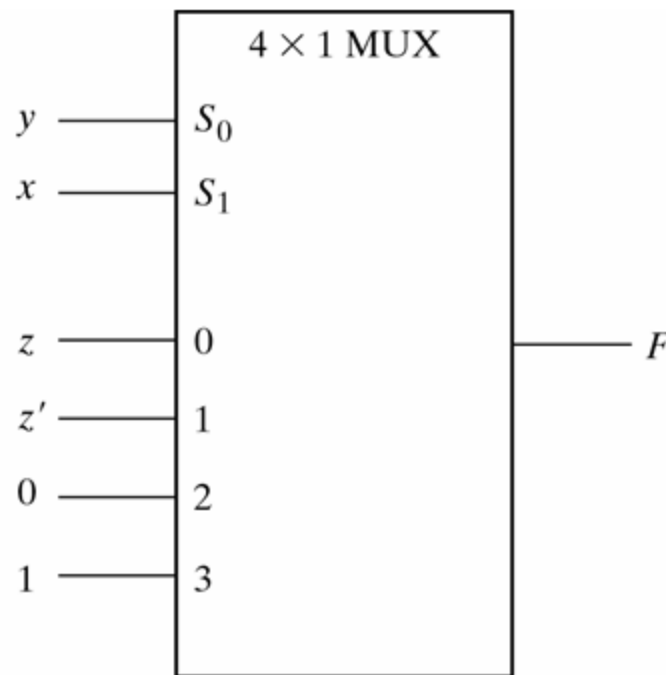
مراحل پیاده سازی:

- پایینترین متغیر را جدا کرده، بقیه را به ترتیب به ورودیهای انتخاب MUX وصل کنید.
- جدول درستی تابع را تشکیل دهید.
- محتوای جدول درستی را از بالا دو سطر، دو سطر جدا کنید.
- برای قسمتهای جدا شده، اگر هر دو مقدار تابع برابر با:
 - صفر هستند، آنگاه ورودی متناظر از MUX برابر با 0 قرار داده می شود.
 - 1 هستند، آنگاه ورودی متناظر از MUX برابر با 1 قرار داده می شود.
 - مقادیر متغیر منفرد هستند، آنگاه همان متغیر منفرد به ورودی متناظر از MUX داده می شود.
 - مکمل مقادیر متغیر منفرد هستند، آنگاه مکمل متغیر منفرد به ورودی متناظر از MUX داده می شود.

پیاده سازی توابع با استفاده از مالتی پلکسر

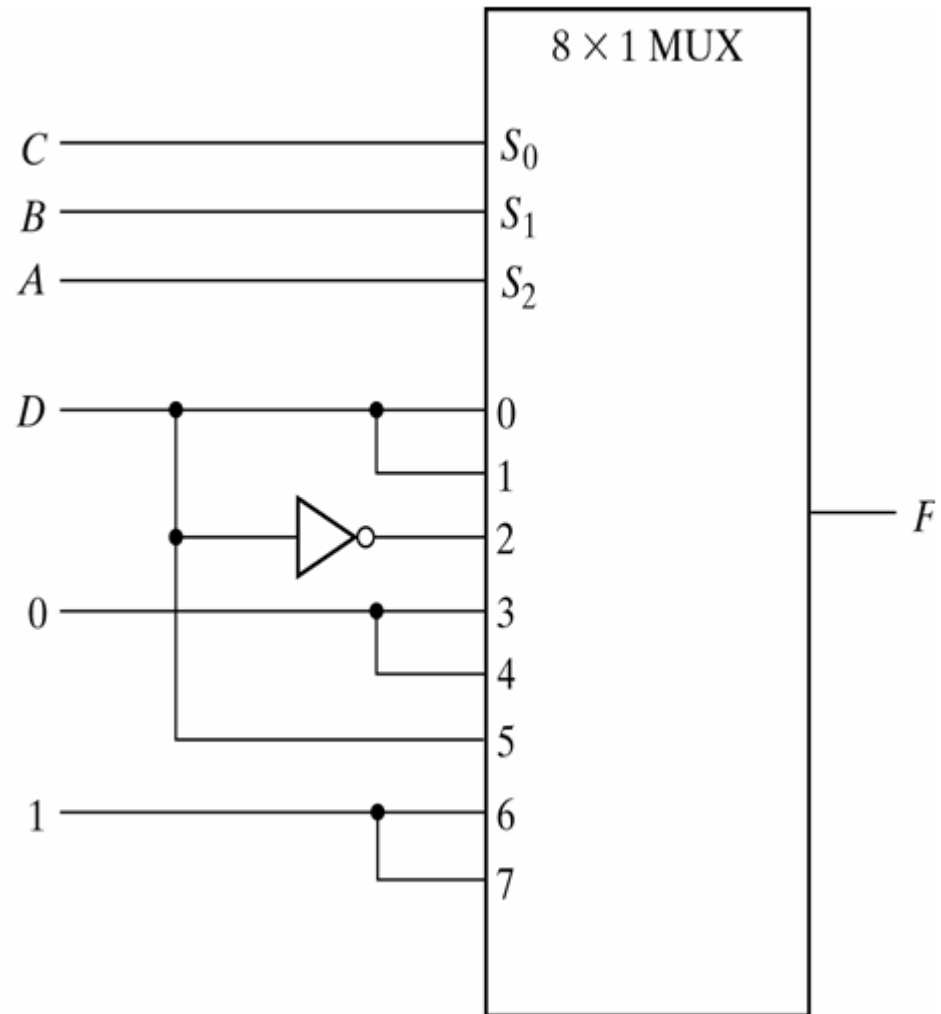
- جدول درستی تابع بولی زیر همراه با نحوه پیاده سازی آن توسط MUX نشان داده شده است:

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	



پیاده سازی توابع با استفاده از مالتی پلکسر

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>F</i>	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

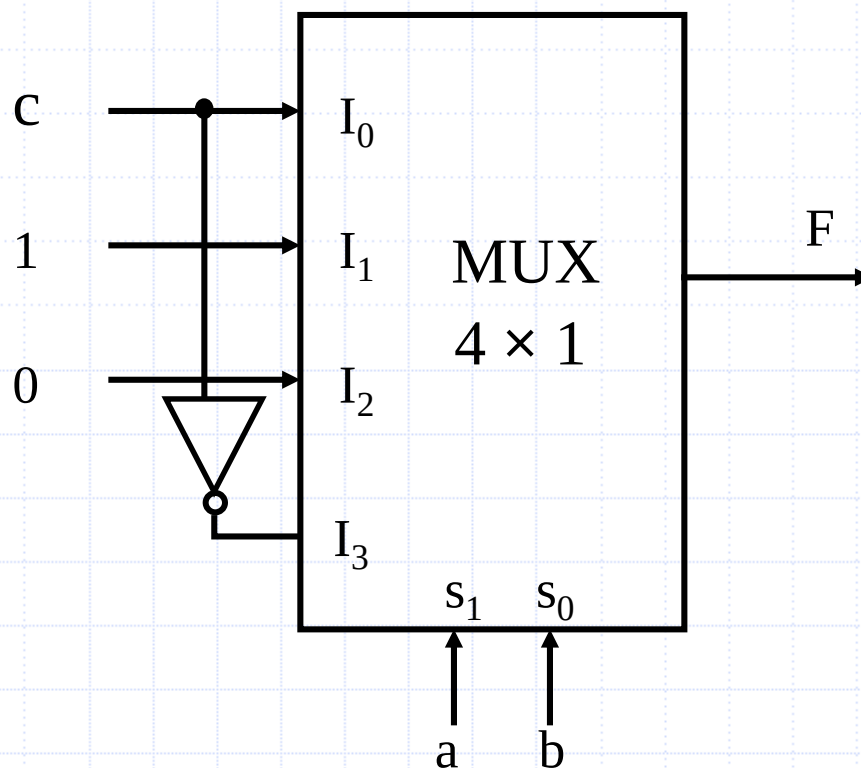


پیاده سازی توابع با استفاده از مالتی پلکسر

مثال: ✓

$$F(a, b, c) = \sum m(1, 2, 3, 6)$$

a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0



پیاده سازی توابع با استفاده از مالتی پلکسر

✓ مثال پیاده سازی تابع ۳ متغیره با مالتی پلکسر ۲ به ۱:

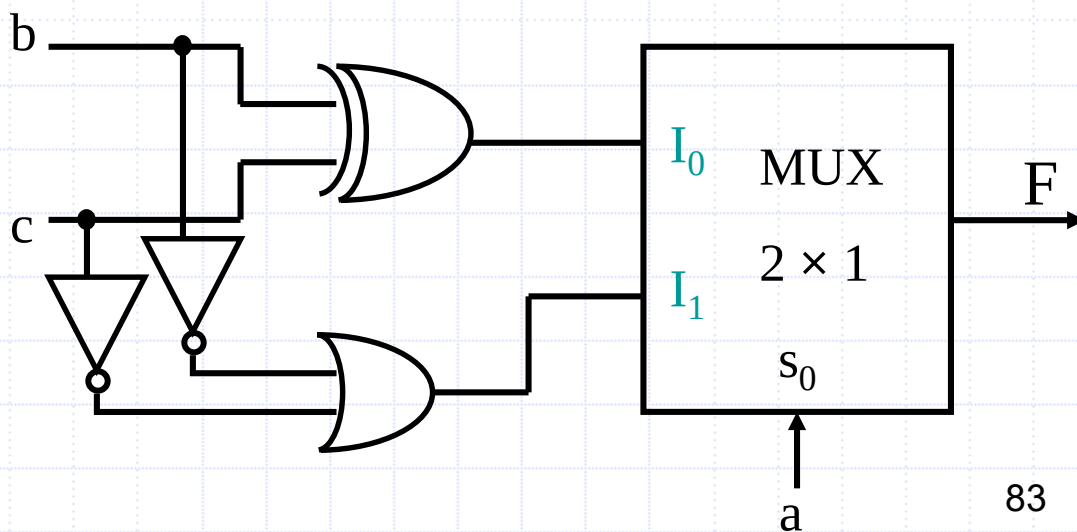
$$F(a, b, c) = \sum m(1, 2, 4, 5, 6)$$

- I_0 و I_1 را به صورت تابعی از b و c در نظر میگیریم

a	b	c	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

$$\begin{cases} I_0 = b \oplus c \\ I_1 = \bar{b} + \bar{c} = \overline{bc} \end{cases}$$

b c	00	01	11	10
a 0	0	1	0	1
a 1	1	1	0	1



پیاده سازی توابع با استفاده از مالتی پلکسر

✓ مثال پیاده سازی تابع ۴ متغیره با مالتی پلکسر ۴ به ۱:

$$F(a, b, c, d) = \sum m(1, 3, 5, 6, 7, 10, 11, 15)$$

I_0, I_1, I_2, I_3 را به صورت تابعی از c و d در نظر می گیریم:

a	b	c	d	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

I_0

I_1

I_2

I_3



$$\begin{cases} I_0 = d \\ I_1 = d + c \\ I_2 = c \\ I_3 = cd \end{cases}$$

		c d			
		00	01	11	10
a b	00	0	1	1	0
	01	0	1	1	1
	11	0	0	1	0
	10	0	0	1	1

I_0

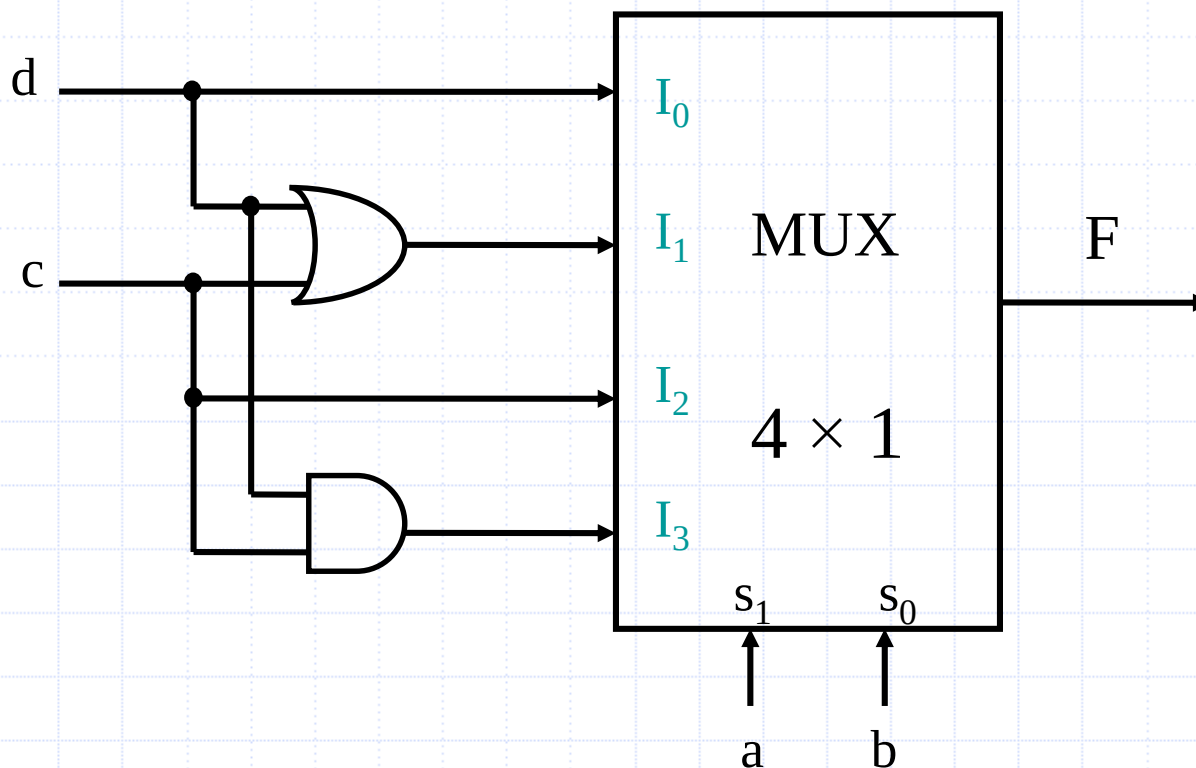
I_1

I_3

I_2

پیاده سازی توابع با استفاده از مالتی پلکسر

✓ ادامه مثال: $F(a, b, c, d) = \sum m(1, 3, 5, 6, 7, 10, 11, 15)$



دی مالتی پلکسر (DeMultiplexer)

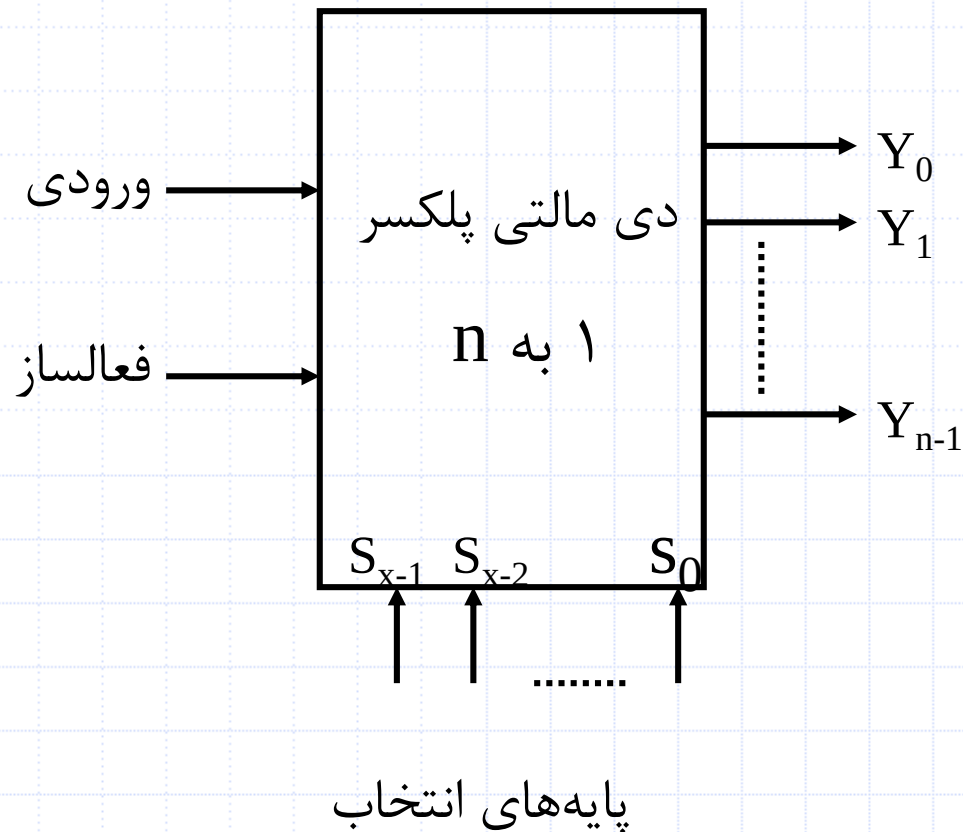
✓ یک مدار منطقی ترکیبی که یک خط ورودی را به یکی از n خط خروجی وصل می کند.

✓ خط خروجی خاص با یک کد انتخاب X بیتی معین می شود که:

$$2^x \geq n$$

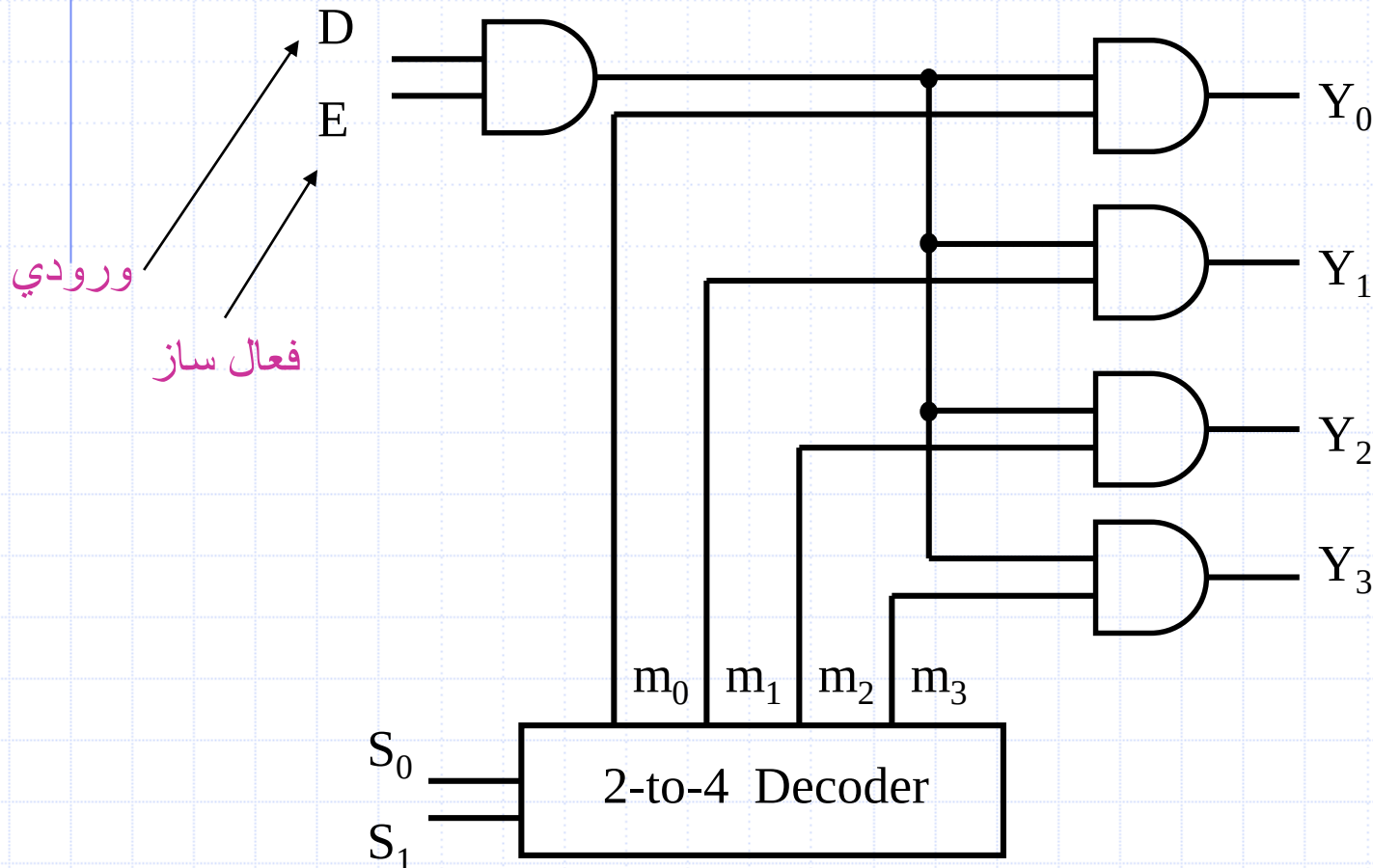
✓ در این حالت کد انتخاب برای تولید مینترم های X بیتی بکار می رود.

دی مالتی پلکسر (DeMultiplexer)



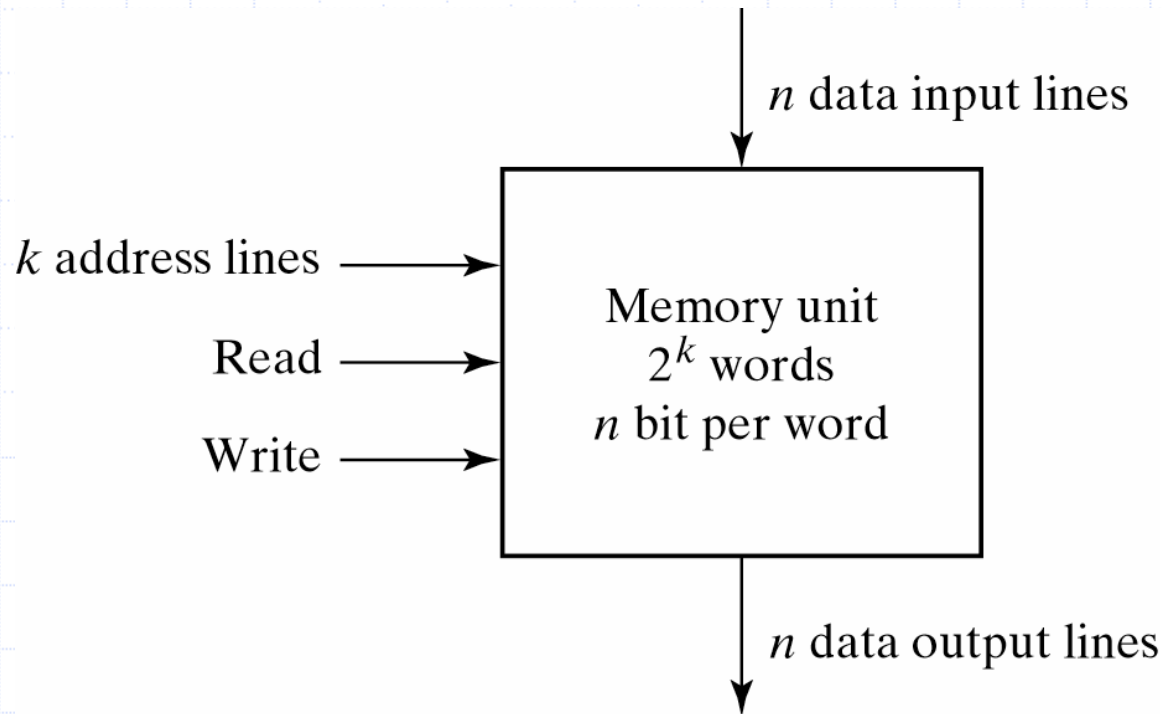
دی مالتی پلکسر (DeMultiplexer)

✓ ساختار داخلی دی مالتی پلکسر ۱ به ۴ با فعال ساز



حافظه ها

✓ بلوک دیاگرام کلی حافظه



حافظه ها

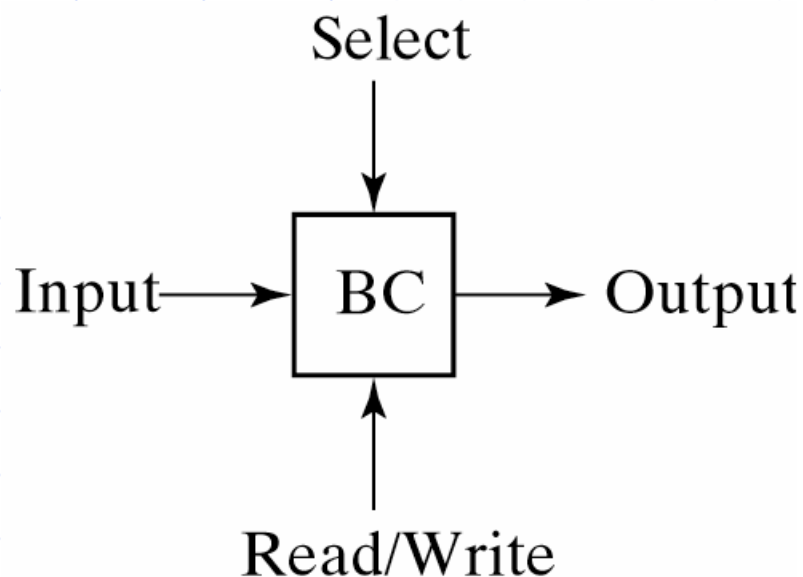
✓ یک حافظه 1024×16

Memory address		Memory content
Binary	decimal	
0000000000	0	1011010101011101
0000000001	1	1010101110001001
0000000010	2	0000110101000110
	•	•
	•	•
	•	•
1111111101	1021	1001110100010100
1111111110	1022	0000110100011110
1111111111	1023	1101111000100101

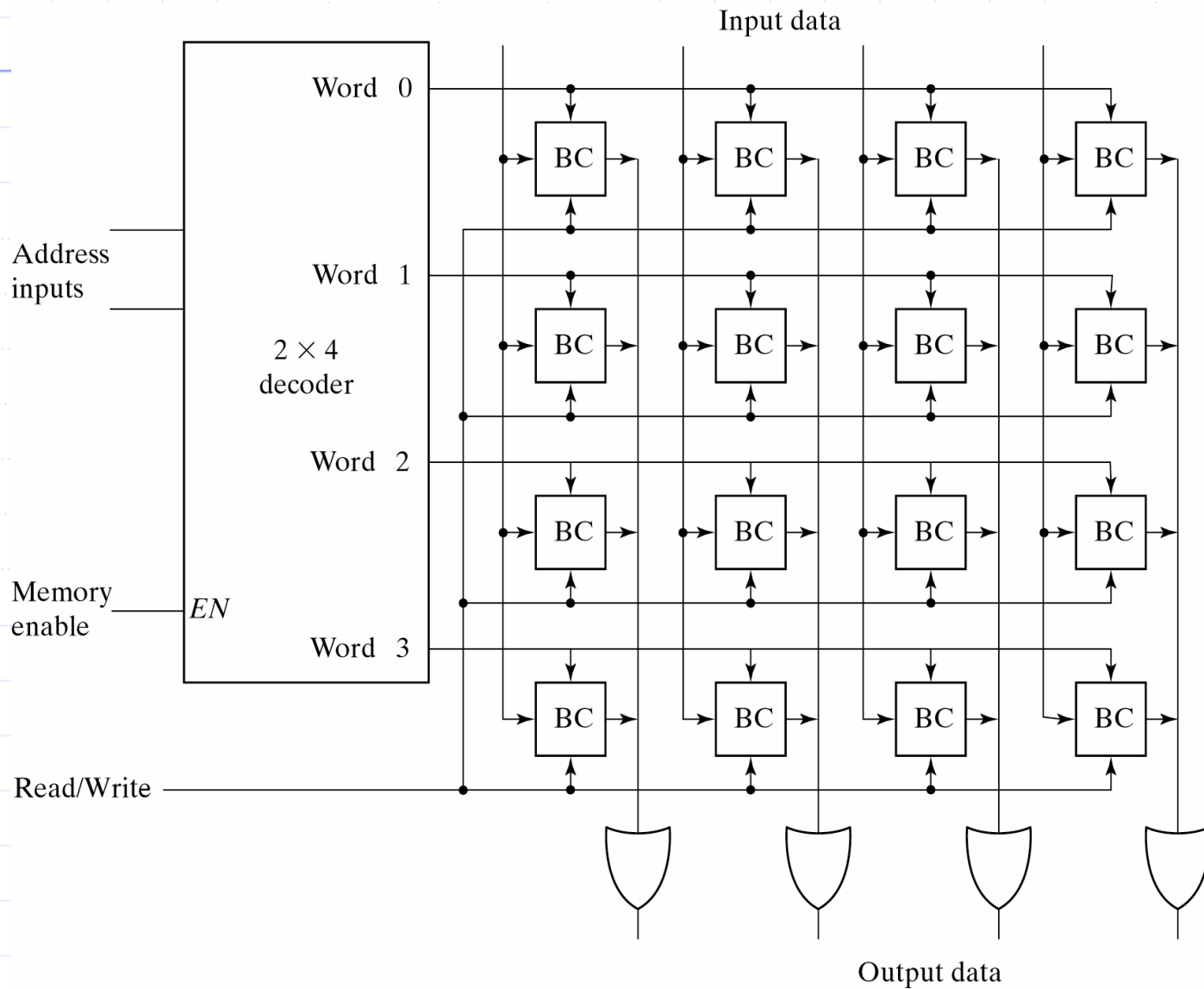
حافظه ها

سلول حافظه:

- هر حافظه از تعدادی اجزای کوچکتر ساخته شده است که سلولهای حافظه نامیده می شوند.



حافظه ها

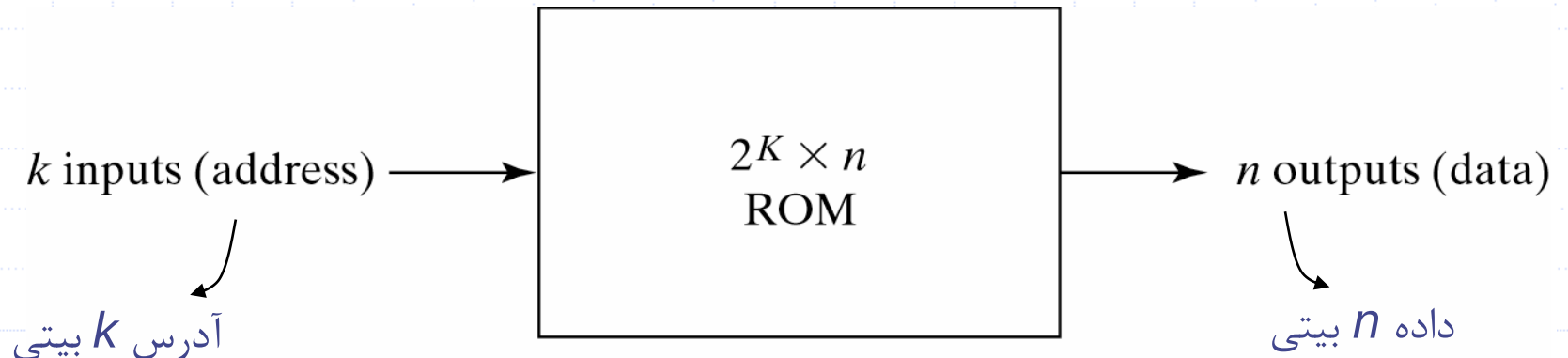


حافظه ROM (Read Only Memory)

- حافظه های فقط خواندنی: جزو مدارهای ترکیبی (نه ترتیبی)
- ROM های اولیه تنها یکبار نوشته می شوند و دائم خوانده می شوند
- دارای انواع متفاوتی است:
- Factory Programmed or Mask ROM
- Field Programmable:
 - PROM: Programmable ROM
 - EPROM: Erasable PROM
 - EEPROM: Electrically Erasable PROM
 - EAROM: Electrically Alterable ROM
 - Flash Memory

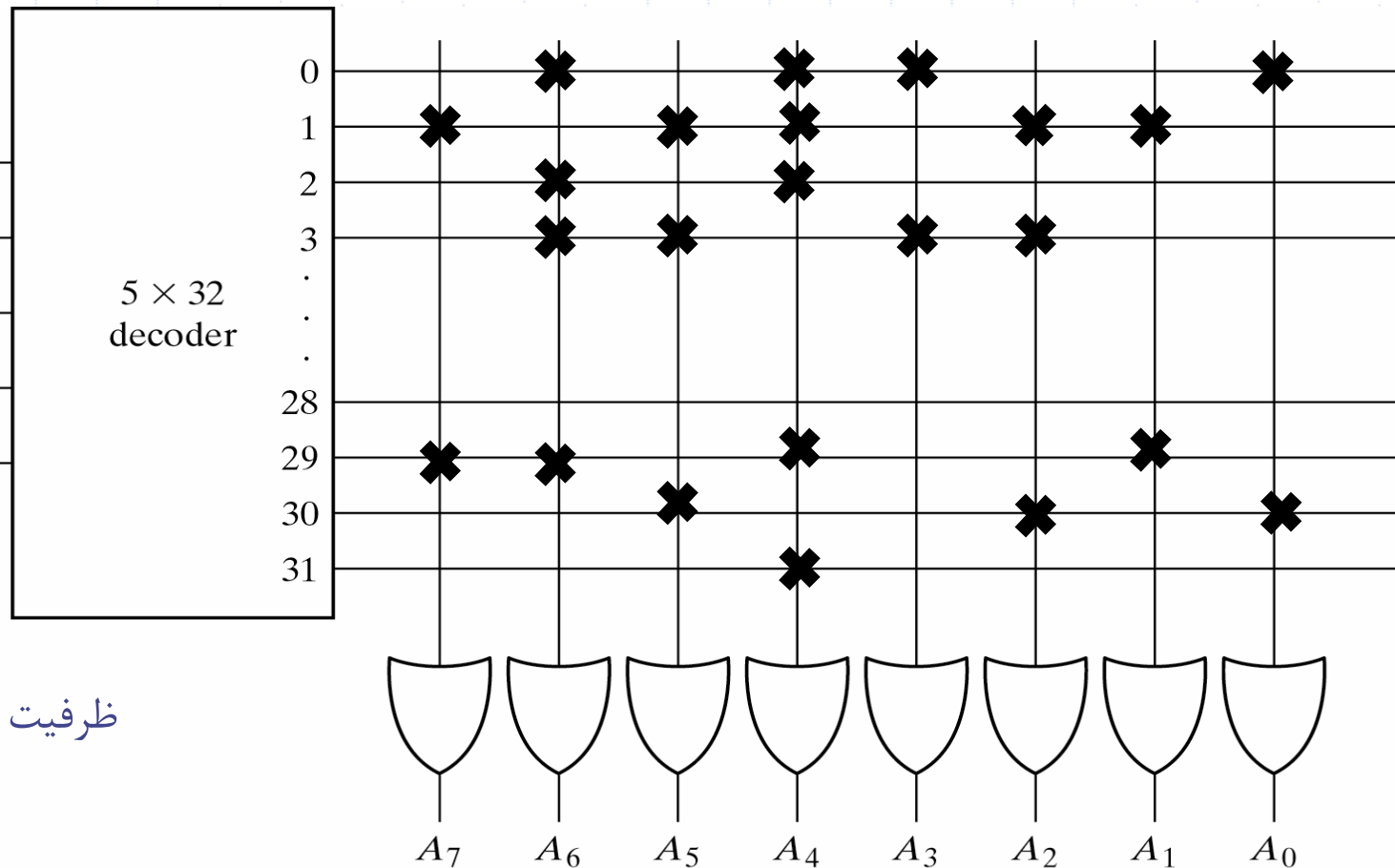
حافظه ROM (Read Only Memory)

✓ بلوک دیاگرام یک ROM



حافظه ROM (Read Only Memory)

ساختار داخلی یک ROM ۸*۳۲



آدرس ۵ بیتی

ظرفیت حافظه: $2^5 = 32$

داده ۸ بیتی

حافظه ROM (Read Only Memory)

از دو دیدگاه می‌توان به ROM برنامه‌ریزی شده نگاه کرد

- چندیدن تابع را از روی ورودیها می‌سازد که بسته به ترکیب ورودیها، هر تابع 0 یا 1 خواهد شد.
- در هر ترکیب خاص از ورودیها، مقادیر توابع را کنار هم، می‌توان به عنوان داده خروجی در نظر گرفت. مثل این است که ترکیب ورودیها، آدرس خاصی را مشخص می‌کند و مقادیر توابع کنار هم، بیتیهای داده ذخیره‌شده در آن آدرس را نشان می‌دهد.

حافظه ROM (Read Only Memory)

پیاده‌سازی توابع با استفاده از حافظه ROM:

- با توجه به ساختار داخلی ROM مشخص می‌شود که قبل از برنامه ریزی اتصالات، هر خروجی حاصل جمع تمام مینترم‌های n متغیر ورودی را فراهم می‌کند و لذا مقدار 1 دارد.

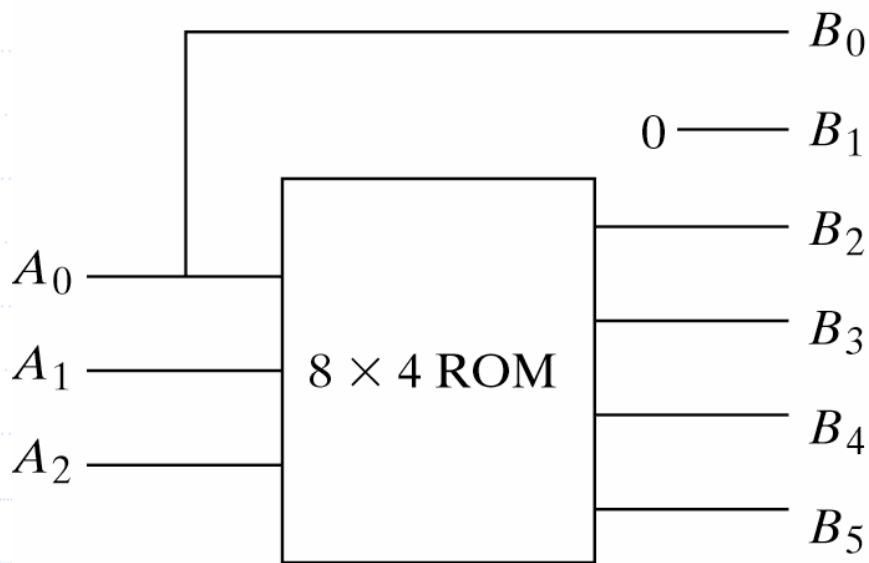
- با برنامه‌ریزی مناسب (با انتخاب مناسب مینترم‌ها) می‌توان هر تابعی را پیاده‌سازی کرد.

حافظه ROM (Read Only Memory)

- مثال: با استفاده از ROM مداری طراحی کنید که ورودی آن یک عدد ۳ بیتی و خروجی آن عدد دودویی است که برابر با مجذور عدد دودویی ورودی است.

A_2	A_1	A_0	B_5	B_4	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1
0	1	0	0	0	0	1	0	0
0	1	1	0	0	1	0	0	1
1	0	0	0	1	0	0	0	0
1	0	1	0	1	1	0	0	1
1	1	0	1	0	0	1	0	0
1	1	1	1	1	0	0	0	1

حافظه ROM (Read Only Memory)



(a) Block diagram

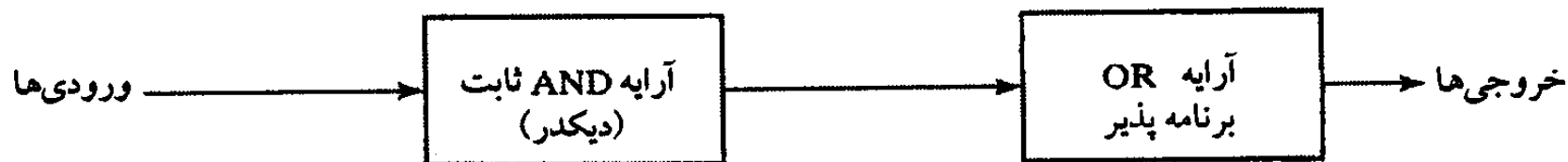
A_2	A_1	A_0	B_5	B_4	B_3	B_2
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	1
0	1	1	0	0	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	1	0	1	0	0	1
1	1	1	1	1	0	0

(b) ROM truth table

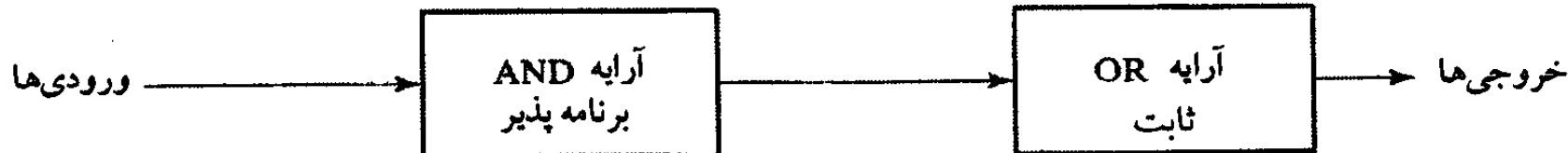
قطعات دیجیتال قابل برنامه‌ریزی (Programmable Logic Device (PLD))

- ROM
- PLA (Programmable Logic Array)
- PAL (Programmable Array Logic)
 - GAL (Generic Array Logic): several PALs; erasable and reprogrammable
- CPLD (Complex PLD)
 - Thousands or hundreds of thousands logic gates
- FPGA (Field-Programmable Gate Array)
 - More complex than CPLD
 - Consists of look-up tables, multiplexers, gates and flip-flops

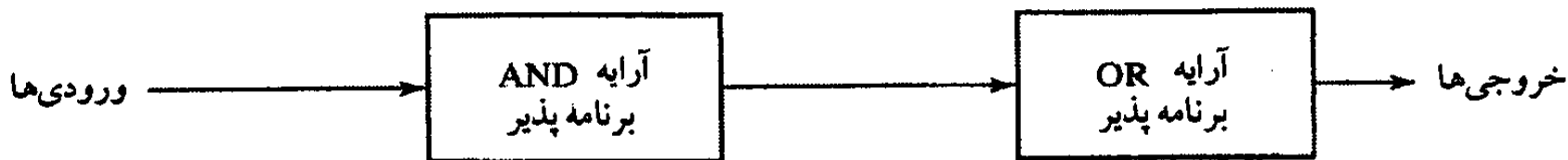
قطعات دیجیتال قابل برنامه‌ریزی (Programmable Logic Device (PLD))



(الف) حافظه فقط خواندنی برنامه پذیر (PROM)



(ب) منطق آرایه‌ای برنامه پذیر (PAL)



(پ) آرایه منطقی برنامه پذیر (PLA)

PLA

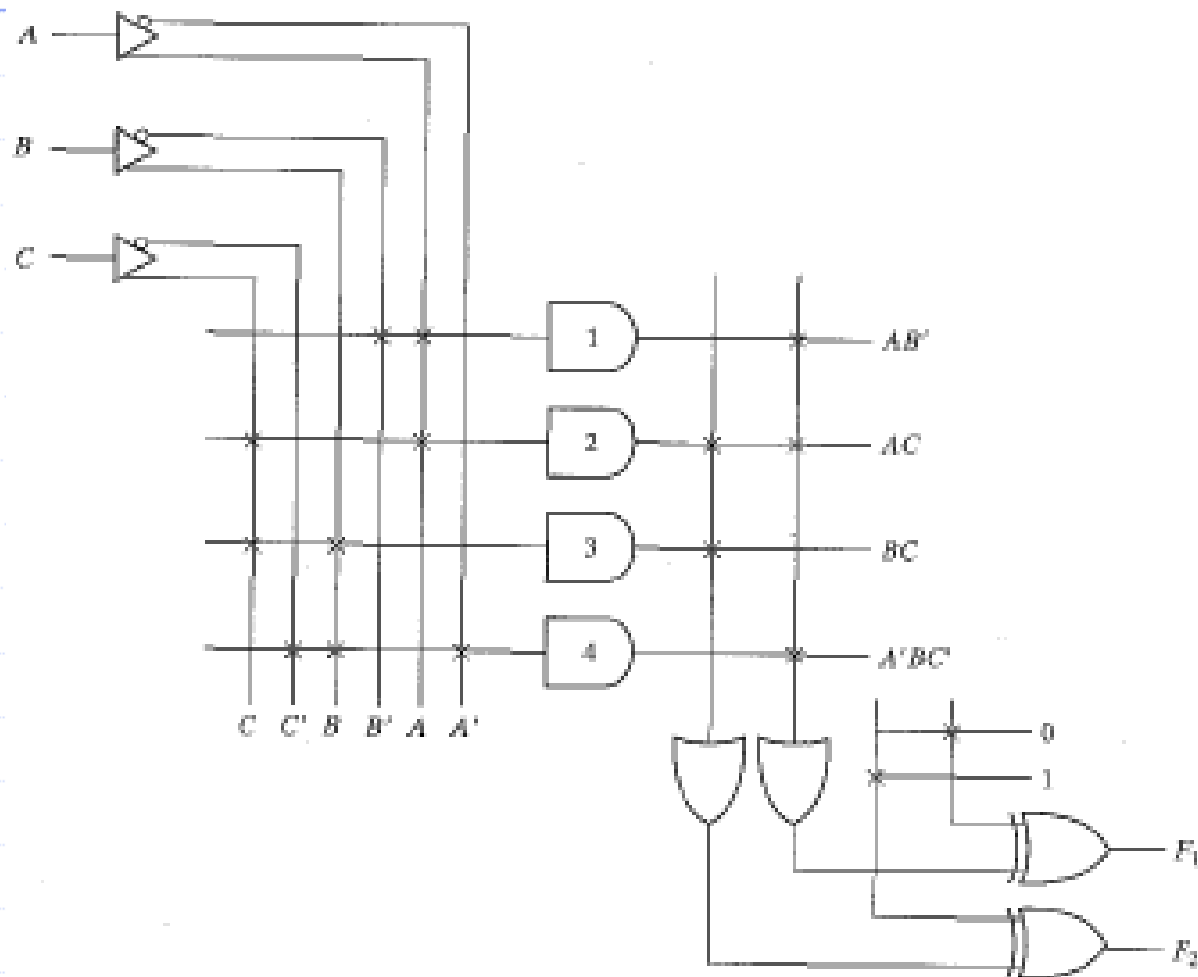


FIGURE 7-14
PLA with 3 Inputs, 4 Product Terms, and 2 Outputs

PAL

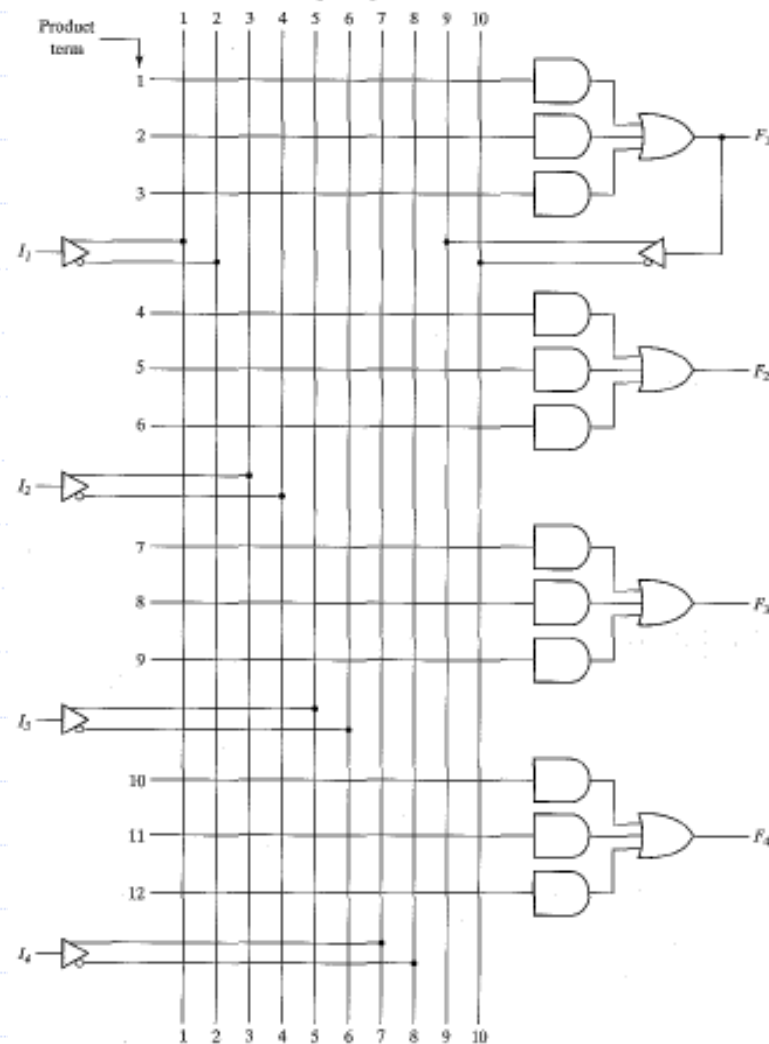


FIGURE 7-16
PAL with Four Inputs, Four Outputs, and Three-Wide AND-OR Structure

CPLD

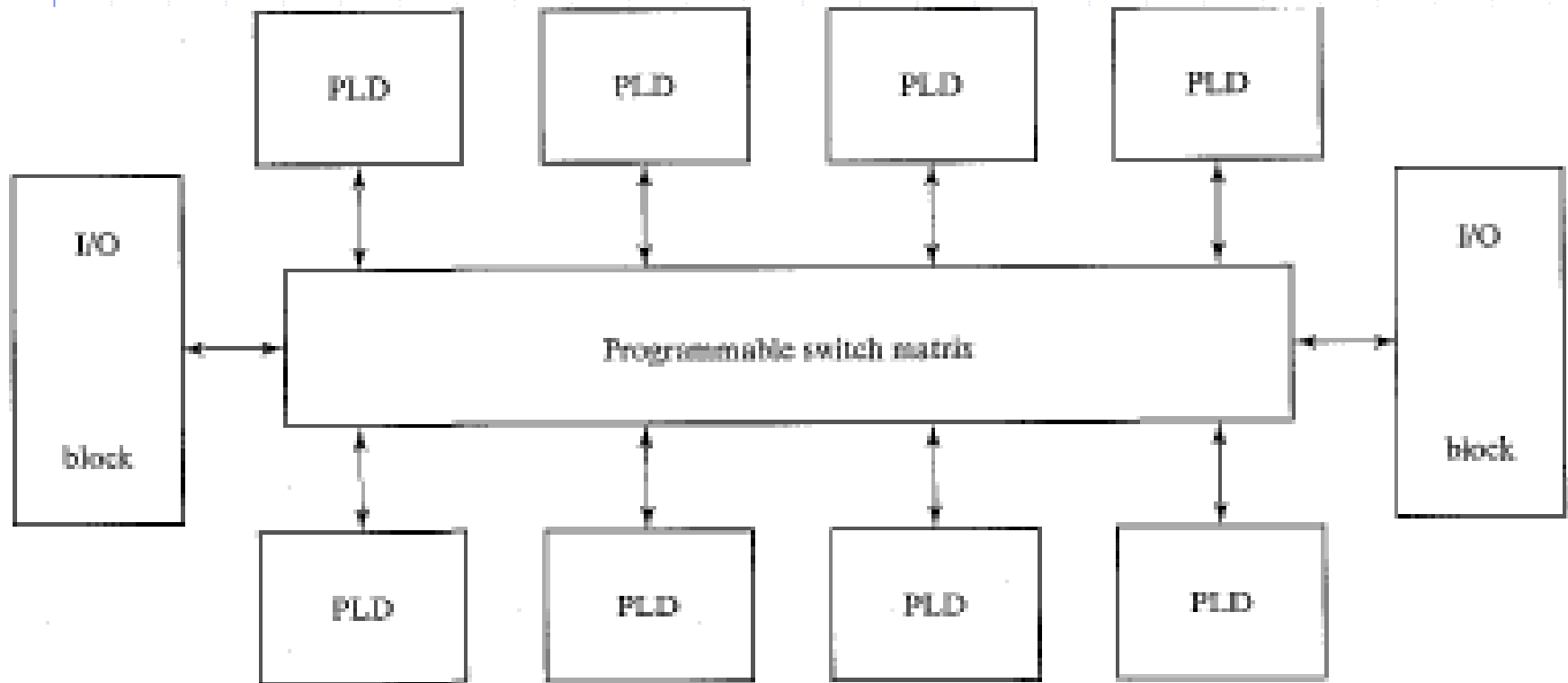


FIGURE 7-20
General CPLD Configuration